

Big Data and Cloud Computing

Pedro Afonso Nunes Fernandes – 202207987
Ângelo Henrique Faria Sá Brás – 202509789



Masters in Information Security

May, 2026

Contents

1	Introduction	2
2	Deployment Environment and Installation	4
2.1	Experimental Environment and Machine Specifications	4
2.2	Installation and Bootstrap Procedure	5
3	System Architecture	7
3.1	Component Overview	8
3.2	Request Routing	9
3.3	Authentication Design	9
3.4	Multi-Tenancy and Isolation	9
3.5	Asynchronous and Concurrent Execution	9
3.6	Technology Stack	10
4	Authentication and Multi-Tenancy	10
4.1	Local User Model	10
4.2	OpenNebula Account Synchronisation	11
4.3	Registration Flow	12
4.4	Login and Token Issuance	12
4.5	Request Authentication via JWT Dependency	13
4.6	Profile Management Endpoints	13
4.7	Multi-Tenancy Enforcement Summary	14
5	Virtual Machine Management	14
5.1	OpenNebula Connection Layer	15
5.2	VM Lifecycle Operations	15
5.3	Compute Router Endpoints	16
5.4	Resource Monitor	17
5.5	SLA Policy Constants	17
5.6	WebSocket SSH Terminal	18
6	Storage Services	19
6.1	Object Storage: MinIO Integration	19
6.2	Object Lifecycle Operations	19
6.3	Object Storage API Endpoints	20
6.4	Block Storage: OpenNebula Datablock Images	20
6.5	Block Storage API Endpoints	22
7	Container Management Service	22
7.1	Docker SDK SSH Integration	22
7.2	User Isolation via Docker Labels	22
7.3	Dynamic Port Assignment	23
7.4	Container Lifecycle Operations	23
7.5	Self-Healing Docker Installation	25
7.6	Container API Endpoints	25

8 Database-as-a-Service	25
8.1 Database Provisioning	25
8.2 Container Discovery Across VMs	27
8.3 Database Metrics	28
8.4 Database Lifecycle Operations	28
8.5 Database API Endpoints	28
8.6 Clustered PostgreSQL with Streaming Replication	28
9 Autoscaling and Load Balancing	29
9.1 VM-Level Autoscaler	29
9.2 Container-Level Autoscaler	33
9.3 Nginx-Based Container Load Balancer	34
9.4 HAProxy Database Load Balancer	35
9.5 Concurrency Model	37
10 Web Dashboard	38
10.1 Application Structure	38
10.2 HTTP Client and JWT Interceptor	38
10.3 Authentication State Management	39
10.4 Real-Time Metrics with React Query	39
10.5 Overview Dashboard	40
10.6 VM Management Page	40
10.7 Containers and Storage Pages	40
11 Tests & Evaluation Metrics	40
11.1 Quick Summary	41
11.2 Test 1 — OpenNebula Connection Integrity	41
11.3 Test 2 — Block Storage Lifecycle	42
11.4 Test 3 — Scale-to-Zero & Wake-on-API	43
11.5 Test 4 — HTTP Load Balancer CPU Stress Test	44
11.6 Test 5 — Clustered Database & HAProxy L4 Load Balancing	46
11.7 Evaluation Metrics	47
11.8 Infrastructure State at Test Time	49
12 Conclusion	49
12.1 Engineering Achievements	49
12.2 Limitations and Future Work	50
12.3 Closing Remarks	51

1 Introduction

Cloud computing has fundamentally transformed how computational resources are provisioned, managed, and billed. Rather than purchasing and maintaining dedicated physical servers, organisations now lease compute, storage, and platform capabilities on demand from a cloud provider. This shift towards Infrastructure-as-a-Service (IaaS) and Platform-as-a-Service (PaaS) abstractions has enabled elastic scaling, high availability, and unprecedented resource efficiency. However, commercially operated public clouds obscure the internal mechanisms responsible for these properties. Understanding how such systems are engineered , from the hypervisor layer upwards , requires building one from scratch.

In this project we designed and implemented a full-stack, multi-tenant cloud management platform on top of **OpenNebula 7.2.0** [1], an open-source cloud orchestrator that exposes a QEMU/KVM hypervisor via an

XML-RPC interface. Our platform wraps OpenNebula with a RESTful API built in Python using **FastAPI** [2], adds higher-level PaaS abstractions such as containerised workloads and Database-as-a-Service (DBaaS), and presents all capabilities through a **React 18** single-page application dashboard. The result is a cohesive cloud manager with feature parity to reduced-scale versions of Amazon Web Services EC2, S3, RDS, and ECS.

Goals

The primary goal of this project is to implement a coherent set of cloud services, from raw virtual machine provisioning to fully automated workload scaling, while enforcing strict multi-tenant isolation and energy-saving policies. Concretely, we set out to build the following subsystems.

User Management & Authentication. A JWT-based authentication system that issues 24-hour bearer tokens and bidirectionally synchronises user accounts with the underlying OpenNebula instance, so that every API user maps to a real **oneadmin**-managed identity with appropriate resource ownership.

Elastic Compute. On-demand virtual machine provisioning using OpenNebula’s XML-RPC `template.instantiate` interface, augmented by a *hot standby pool* of pre-warmed VMs that reduces provisioning latency from approximately 90 seconds to under one second.

Block Storage. Dynamic allocation of OpenNebula datablock images that can be hot-plugged into running VMs using `virsh attach-disk` semantics, formatted with `ext4`, and mounted persistently.

Object Storage. A per-user, S3-compatible object store backed by a locally deployed MinIO [3] server, with automatic bucket creation tied to user registration.

Container Service. On-demand Docker container management running directly on user-provisioned VMs, with per-user label-based isolation (`ccloud_user=<username>`), dynamic host-port assignment, and parallel SSH-based querying via `ThreadPoolExecutor`.

Database-as-a-Service. Automated provisioning of `postgres:16-alpine` container instances, each configured with a randomly generated 24-character password, a volume-backed data directory, and a full connection DSN delivered to the user on creation.

Load Balancing & Clustering. An Nginx-based HTTP load balancer for scaled container groups and an HAProxy [4] Layer 4 TCP load balancer for PostgreSQL clusters, including physical streaming replication via `pg_basebackup` and WAL log shipping.

SLA Enforcement & Autoscaling. A background `threading.Thread` that enforces CPU-based scale-up (> 70% threshold) and scale-down (< 20% threshold), scale-to-zero inactivity suspension after two hours of API silence, and a container-level autoscaler with a 10-second check cycle, 30-second cooldown, and 60-second stabilisation window.

Energy Accounting. An endpoint (`GET /compute/energy-stats`) that computes VM-hours saved relative to a static full-capacity baseline, translating hours into kilowatt-hours and estimated CO₂ savings using a 50 W/VM draw and a 0.4 kg CO₂/kWh grid emission factor.

Web Dashboard. A React + Vite + Tailwind CSS management interface with live CPU charts, WebSocket-based in-browser SSH terminals, TanStack React Query auto-refresh, and Zustand token state management.

Report Structure

The remainder of this report is organised as follows. Section 2 details the deployment environment and installation procedure. Section 3 describes the overall layered system architecture and the key design decisions. Section 4 explains the authentication and multi-tenancy model. Section 5 details the virtual machine management

subsystem, including the OpenNebula connection layer, VM lifecycle operations, block storage, the WebSocket SSH terminal, and the SLA monitor. Section 6 details the MinIO object storage integration. Section 7 describes the Docker-based container service. Section 8 presents the Database-as-a-Service layer, including replica clustering. Section 9 covers the autoscaling and load balancing subsystems at both the VM and container levels, and explains the concurrency model. Section 10 describes the React dashboard. Section 11 presents the tests carried out and the evaluation metrics collected. Section 12 concludes with a reflection on engineering challenges and directions for future work.

2 Deployment Environment and Installation

This section details the deployment topology, hardware and virtualization specifications, and step-by-step installation instructions for both the local cloud manager service and the remote hypervisor infrastructure.

2.1 Experimental Environment and Machine Specifications

The platform operates across a two-node architecture bridged by a secure SSH port-forwarding tunnel. The API gateway and web UI dashboard are deployed on a local workstation, while the core virtualization orchestrator, hypervisor, and object storage daemon reside on a single remote host machine (**PonchaLaptop**). Guest virtual machines run on this host machine within a private virtual network.

2.1.1 Remote Host Machine (PonchaLaptop)

The remote host machine, **PonchaLaptop**, acts as the primary cloud controller and virtualization node. It runs Ubuntu Server 22.04 LTS and hosts the OpenNebula daemon (**oned**), the KVM/QEMU hypervisor driver, and the MinIO object storage server. Because the local backend communicates with OpenNebula and MinIO over SSH port forwarding, **PonchaLaptop** also serves as the SSH tunnel target. Its hardware and virtualization characteristics are detailed in Table 1.

Table 1: Remote Host Machine (**PonchaLaptop**) Specifications.

Specification	Value
Operating System	Ubuntu Server 24.04 LTS (x86_64)
Hypervisor Driver	KVM / QEMU
Processor	Intel Core i5 family (4 physical cores)
Random Access Memory	16 GB DDR4
Storage System	500 GB Solid State Drive (SSD)
Private Network Bridge	172.16.100.* (host-only VLAN bridge)
Hosted Core Services	OpenNebula version 7.2.0, MinIO S3 Server

2.1.2 Local Workstation

The local workstation represents the administrator’s development machine. It runs the FastAPI Python backend and the Vite-powered React dashboard. All interactions with the remote host are proxied through local forwarded ports. The workstation specifications are detailed in Table 2.

Table 2: Local Workstation Specifications.

Specification	Value
Operating System	macOS / Linux
Security Tunnel	SSHv2 Client (with key-based SSH authentication)
Local API Ports	8000 (FastAPI), 5173 (React Dashboard)
Local Forwarded Ports	2633 (XML-RPC), 9002 (MinIO API), 9003 (MinIO Console)

Secure Network Connectivity via Tailscale. To bridge the local workstation and the remote virtualization host (PonchaLaptop) across separate physical networks and NAT (Network Address Translation) boundaries, we utilized *Tailscale*, a zero-configuration peer-to-peer overlay network built on the WireGuard VPN protocol. Tailscale assigned static, private IPv4 addresses within the CGNAT range (100.64.0.0/10) to both nodes. This established a secure encrypted tunnel between the two endpoints without requiring public IP addresses, dynamic DNS setup, or home router port forwarding, allowing the SSH port-forwarding tunnels to connect reliably across any network environment.

2.1.3 Guest Virtual Machine Nodes (Compute Instances)

The compute instances provisioned dynamically by tenants are minimized Alpine Linux VM domains hosted on PonchaLaptop’s KVM hypervisor. The specifications for these guest instances are detailed in Table 3.

Table 3: Guest Compute Instance Specifications.

Specification	Value
Operating System Image	Alpine Linux 3.20 (minimized footprint)
Virtual Processors	1 vCPU
Allocated RAM	2048 MB
Root Disk Size	4 GB (expandable dynamically)
Bootstrap Agent	one-context (ISO mounting on boot)
Exposed Services	Docker Engine (daemon socket over SSH port 22)

2.2 Installation and Bootstrap Procedure

Bootstrapping the cloud management framework requires deploying the remote OpenNebula environment, establishing secure connection tunnels, configuring environment credentials, and starting local API and S3 services.

2.2.1 Step 1: Remote OpenNebula & MinIO Deployment

On the remote host machine (PonchaLaptop), OpenNebula version 7.2.0 is deployed using the official single-node bootstrap installer *MiniOne*. Run the following commands to initialize the hypervisor node:

```

1 # Download the MiniOne bootstrap script
2 wget https://raw.githubusercontent.com/OpenNebula/minione/master/minione
3
4 # Execute MiniOne to deploy frontend + KVM hypervisor node
5 sudo bash minione --version 7.2.0 --username oneadmin --password your_password

```

Listing 1: Deploying OpenNebula using MiniOne.

Upon completion, the OpenNebula daemon binds to local loopback port 2633 and Sunstone Web UI exposes port 80. Additionally, the MinIO binary is downloaded to the home directory and started in the background using port 9002 for the API and port 9003 for the console, persisting its data in the local `./minio_data` folder:

```

1  # Start MinIO locally on PonchaLaptop
2  wget https://dl.min.io/server/minio/release/linux-amd64/minio
3  chmod +x minio
4  sudo mv minio /usr/local/bin/
5
6  # Change the path of minio in start_minio.sh to /usr/local/bin/
7  nano start_minio.sh
8
9  # Start minio
10 bash start_minio.sh

```

Listing 2: Starting MinIO on PonchaLaptop.

2.2.2 Step 2: Establish the Secure SSH Tunnel

Because the XML-RPC endpoint and the MinIO storage daemon bind only to localhost addresses on PonchaLaptop for security, a secure forwarding tunnel must be initialized from your local workstation:

```

1  ssh -L 8080:localhost:80 -L 2633:localhost:2633 -L 9002:localhost:9002 -L
    9003:localhost:9003 ubuntu@[ponchalaptop-ip]

```

Listing 3: Establishing SSH tunnel connections.

This command forwards:

- Port 8080 locally → Sunstone Web UI.
- Port 2633 locally → OpenNebula XML-RPC API.
- Port 9002/9003 locally → MinIO S3 object API and console.

2.2.3 Step 3: Environment Variable Configuration

Create a local configuration file named `.env` in the project root directory on the local workstation to store credentials for OpenNebula, the SSH Gateway tunnel, and the object storage server:

```

1  # OpenNebula Credentials
2  ONE_USER="oneadmin"
3  ONE_PASSWORD="your_password"
4
5  # SSH Gateway details for VM command execution (PonchaLaptop)
6  GATEWAY_IP="ponchalaptop_host_or_ip"
7  GATEWAY_PORT=22
8  GATEWAY_USER="gateway_username"
9  GATEWAY_PASSWORD="gateway_password"
10
11 # MinIO Object Storage configuration
12 MINIO_ENDPOINT="localhost:9002"
13 MINIO_ACCESS="minioadmin"
14 MINIO_SECRET="minioadmin123"
15 MINIO_SECURE="False"

```

Listing 4: Local environment config (`.env`).

2.2.4 Step 4: Python Virtual Environment Setup

On the local workstation, create an isolated Python environment and install the required dependencies (such as the OpenNebula XML-RPC wrapper `pyone`, `fastapi`, `uvicorn`, and `paramiko` for SSH tunneling):

```
1 # Initialize and activate the virtual environment
2 python3 -m venv .venv
3 source .venv/bin/activate
4
5 # Install requirements with pinned dependencies
6 pip install -r requirements.txt"
```

Listing 5: Initializing local python virtual environment.

Note: `bcrypt` is strictly pinned to version 4.0.1 to maintain compatibility with the `passlib` encryption helper.

2.2.5 Step 5: Launch the FastAPI Backend and React Dashboard

With the SSH tunnel active, execute the parallel startup script on the local workstation to start both the Python API server and the React dashboard concurrently:

```
1 # Start FastAPI backend (port 8000) and React dashboard concurrently
2 bash scripts/start_all.sh
```

Listing 6: Starting API gateway and Web UI.

2.2.6 Step 6: Configure Tenant SSH Credentials via the Dashboard

To enable guest virtual machine provisioning and permit remote Docker/database service orchestration, the tenant user must generate a public/private SSH key pair on their local workstation and register the public key in their user profile:

```
1 # Generate a secure Ed25519 SSH key pair
2 ssh-keygen -t ed25519 -C "tenant@example.com" -f ~/.ssh/id_cloudinfra
3
4 # Output the public key to copy
5 cat ~/.ssh/id_cloudinfra.pub
```

Listing 7: Generating a tenant SSH key pair.

Once generated, the public key must be pasted into the User Settings view on the dashboard. The Cloud Manager API processes this input and performs an authenticated user update to store the key inside the tenant's OpenNebula template using a merge operation:

```
1 client.user.update(one_user_id, 'SSH_PUBLIC_KEY = "<public_key>"', 1)
```

Upon subsequent VM provisioning requests, the orchestrator automatically reads this template variable and injects it into the VM's contextualization schema (`SSH_PUBLIC_KEY = "$USER[SSH_PUBLIC_KEY]"`). This authorizes both the tenant developer and the backend orchestrator (connecting over SSH via the `Paramiko` library) to administer services on the node.

The interactive Swagger API documentation is now reachable at `http://localhost:8000/docs`, and the Vite dashboard is serving on `http://localhost:5173`.

3 System Architecture

Our cloud management platform is organised as a four-layer stack. At the base sits the physical hypervisor infrastructure managed by OpenNebula. Above that, a Python FastAPI backend provides a RESTful API that

bridges the OpenNebula XML-RPC interface with the Docker SDK, MinIO SDK, and a local SQLite metadata store. The third layer is a set of higher-level service modules (compute, containers, storage, databases, load balancing) that translate user intent into concrete infrastructure operations. The topmost layer is a React single-page application that renders all service capabilities in a browser. The high-level architectural interactions, layer divisions, and network boundaries are illustrated in Figure 3.

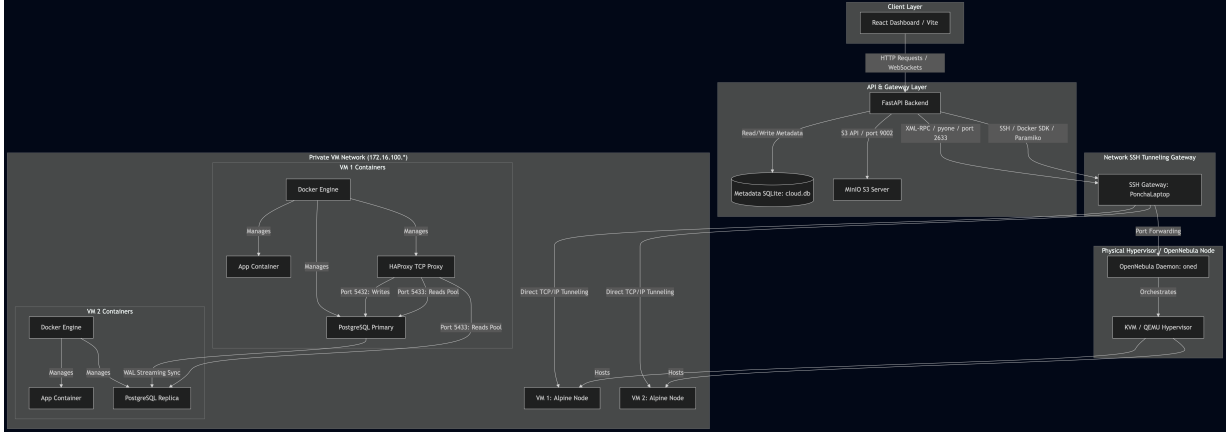


Figure 1: High-Level System Architecture and Component Relationships.

3.1 Component Overview

The following description summarises the component relationships. All client communication, whether from the browser dashboard or an external REST client, enters the system through the FastAPI process, which enforces JWT authentication before dispatching to the appropriate subsystem.

`openebula/connection.py`. Exports `get_client()`, which instantiates an authenticated `pyone.OneServer` [5] pointing to `http://localhost:2633/RPC2`. Because the OpenNebula daemon binds only to localhost on the hypervisor host, connectivity is established through an SSH port-forwarding tunnel. All subsystems obtain a fresh client by calling `get_client()` rather than holding long-lived connections, which avoids XML-RPC session timeouts.

`openebula/vm_manager.py`. Implements the low-level VM primitives: `create_vm`, `destroy_vm`, `get_vm`, `list_all_vms`, `suspend_vm`, and `resume_vm`. VM state is normalised from OpenNebula's integer state codes (e.g., `STATE=3` maps to "ACTIVE") and monitoring data (CPU ticks, memory kilobytes) is converted into human-readable floats.

`openebula/disk_manager.py`. Provides block storage primitives using `client.image.allocate`, `client.vm.attach`, `client.vm.detach`, and `client.image.delete`.

`api/database.py`. Configures a SQLAlchemy [6] engine pointed at a local SQLite file (`cloud.db`). It exposes `get_db()`, a FastAPI dependency that opens a `SessionLocal` per request and closes it in a `finally` block. SQLite was chosen because the metadata it persists (user records, VM ownership mappings, container credentials, disk records) is inherently single-node and does not require the durability or concurrency guarantees of a networked RDBMS.

`api/main.py`. The application entry point. It uses FastAPI's `asynccontextmanager` lifespan to (a) call `Base.metadata.create_all` so all ORM tables are created on startup, (b) run a series of self-healing `ALTER TABLE` statements that add columns introduced in later phases without dropping existing data, (c) start the `AutoScaler` singleton thread (responsible for VM-level scaling and inactivity suspension) and the `ContainerAutoScaler` singleton thread (which manages container-level replica group compliance). On

shutdown, the lifespan calls `autoscaler.stop()` and `container_autoscaler.stop()` to set their stop events.

3.2 Request Routing

Seven FastAPI routers are mounted on the application, each with its own prefix:

Prefix	Module
/auth	api/auth/router.py
/compute	api/compute/router.py
/storage	api/storage/router.py
/containers	api/containers/router.py
/databases	api/database_service/router.py
/loadbalancer	api/loadbalancer/router.py
/terminal	api/compute/terminal.py (WebSocket)

CORS is configured to allow requests from `http://localhost:5173` and `http://localhost:5174` (the Vite development server's default ports), enabling the React dashboard to issue cross-origin API calls during development.

3.3 Authentication Design

Every protected endpoint declares `current_user: User = Depends(get_current_user)` as a parameter. The `get_current_user` dependency in `api/auth/jwt.py` decodes the HS256-signed JWT from the `Authorization: Bearer` header using the `python-jose` library. If decoding succeeds, it queries the `users` table by the `sub` claim (the local user ID), verifies `is_active`, updates `last_active_at` in the same transaction (which the autoscaler reads for inactivity detection), and returns the `User` ORM object. This design means the token never needs to be revoked explicitly: it expires after 24 hours (`TOKEN_EXPIRE_HOURS = 24`), and inactivity is tracked server-side rather than encoded in the token.

3.4 Multi-Tenancy and Isolation

Ownership is enforced at multiple levels to prevent cross-tenant access. At the OpenNebula level, all VMs and disk images are owned by the user's `one_user_id` via `client.vm.chown` and `client.image.chown`. At the container level, every Docker container carries the label `cloud_user=<username>`, and all list, inspect, start, stop, and remove operations filter by this label. At the database level, containers carry `cloud_db_user=<username>`. At the object storage level, each user's files reside in a dedicated MinIO bucket named `user-<username>`, created automatically on first upload. The local SQLite tables (`vm_instances`, `disk_instances`, `db_instances`) all include a `user_id` foreign key referencing `users.id`, and every router query appends a `filter(... user_id == current_user.id)` predicate.

3.5 Asynchronous and Concurrent Execution

FastAPI runs under the Uvicorn ASGI server. Route handler functions that perform I/O-bound operations (SSH connections, Docker SDK calls, OpenNebula XML-RPC) are defined as regular synchronous functions rather than `async def`, so Uvicorn dispatches them to a thread pool via `run_in_executor`, preventing them from blocking the event loop. Within route handlers, the `ThreadPoolExecutor` from Python's `concurrent.futures` is used explicitly whenever a request must query multiple VMs in parallel (see Section 9 for details). Two background autoscaler threads run entirely outside the async event loop as daemon threads, periodically checking cluster state and applying scaling decisions independently of any incoming request.

3.6 Technology Stack

Table 4 summarises the principal software components and their versions as deployed in our experimental environment.

Table 4: Software stack and versions.

Component	Software	Version
Cloud Orchestrator	OpenNebula (oned)	7.2.0
Hypervisor	QEMU/KVM	Host kernel
Backend Language	Python	3.12.3
Web Framework	FastAPI	0.111.0
ORM / Metadata Store	SQLAlchemy / SQLite	2.0.30 / 3.45
OpenNebula SDK	pyone	latest
Container Engine	Docker Engine	26.1.4
Object Storage	MinIO Server	RELEASE.2024-05-10
TCP Load Balancer	HAProxy [4]	2.8.5-alpine
HTTP Load Balancer	Nginx [?]	1.25-alpine
Database Image	PostgreSQL	16-alpine
Auth Cryptography	PyJWT / Passlib (bcrypt)	2.8.0 / 1.7.4
SSH Library	Paramiko	latest
Frontend Framework	React + Vite	18 / 5
CSS Framework	Tailwind CSS	3

4 Authentication and Multi-Tenancy

Authentication in our system operates at two levels simultaneously. At the API level, a JSON Web Token (JWT) [7] gates every protected endpoint. At the infrastructure level, every API user is synchronised with an OpenNebula user account so that all provisioned VMs and disk images carry real OpenNebula ownership, enabling the hypervisor’s own access control model to serve as a second line of isolation enforcement. We call this *dual-layer authentication*.

4.1 Local User Model

The `User` SQLAlchemy model in `api/auth/models.py` is mapped to the `users` table in the local SQLite database. Its columns are:

Column	Purpose
<code>id</code>	Local primary key (integer, auto-increment)
<code>username</code>	Unique login name
<code>email</code>	Unique email address
<code>hashed_password</code>	bcrypt hash of the user’s password
<code>one_user_id</code>	OpenNebula integer user ID
<code>is_active</code>	Soft-disable flag
<code>is_admin</code>	Administrative privileges flag (default: false)
<code>last_active_at</code>	UTC timestamp of last API call (autoscaler inactivity tracking)
<code>created_at</code>	Registration timestamp

The `one_user_id` column is the bridge between the two authentication layers: every operation that calls the OpenNebula XML-RPC API for user-scoped resources passes `current_user.one_user_id` to `chown` operations,

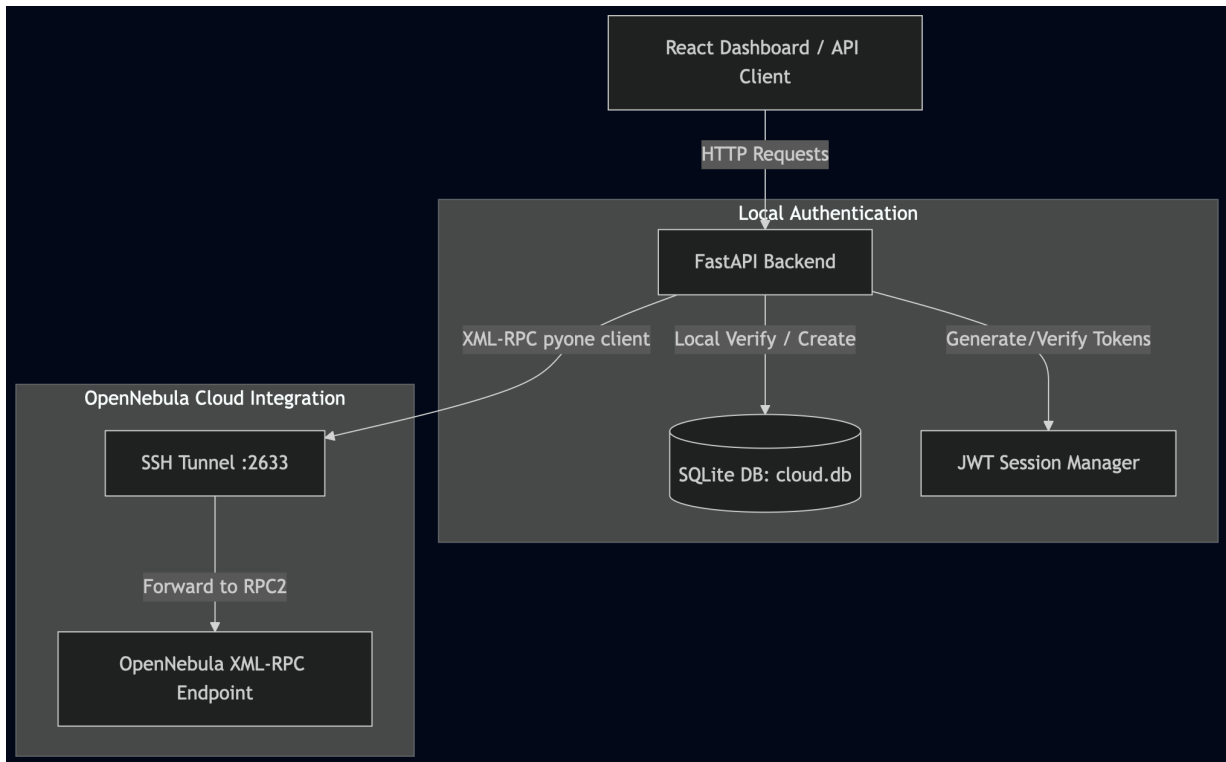


Figure 2: High-Level Auth Architecture .

ensuring that VMs and images are owned at the hypervisor level by the correct user rather than by the system administrator account.

4.2 OpenNebula Account Synchronisation

All OpenNebula interactions for user management are encapsulated in `api/auth/opennebula_sync.py`.

`create_one_user(username, password)`. Allocates a new user in OpenNebula using `client.user.allocate(username, password, "core", [1])`, where "core" selects the SHA256 driver for password hashing within OpenNebula and the array [1] assigns the user directly to Group ID 1 (the unprivileged users group). The allocated `one_user_id` integer is returned and stored in the local `User` record.

`update_one_user_password(one_user_id, new_password)`. Calls `client.user.passwd(one_user_id, new_password)` to update the OpenNebula password in sync with a local password change.

`delete_one_user(one_user_id)`. Calls `client.user.delete(one_user_id)` to remove the account from OpenNebula. This is called *before* removing the local SQLite record, so that if the OpenNebula call fails (e.g., the user still owns running VMs), the local record is preserved and the error propagates to the caller.

`get_one_user_ssh_key(one_user_id)`. Reads the user's SSH public key from the `SSH_PUBLIC_KEY` attribute of their OpenNebula user template via `client.user.info(one_user_id).TEMPLATE.get("SSH_PUBLIC_KEY", "")`.

`update_one_user_ssh_key(one_user_id, public_key)`. Updates the user's SSH key in OpenNebula using `client.user.update(one_user_id, template_string, 1)`, where the `template_string` is `SSH_PUBLIC_KEY=<key>` and the merge mode 1 (MERGE) preserves any other existing template attributes rather than replacing the entire template.

4.3 Registration Flow

The `POST /auth/register` endpoint in `api/auth/router.py` accepts a `UserRegister` schema (username, email, password). Its execution sequence is:

1. Validate that the username and email are not already registered in SQLite (returning HTTP 400 if either exists).
2. Call `create_one_user(username, password)` to allocate the account in OpenNebula. If this step fails (e.g., OpenNebula is unreachable), the registration is aborted and no local record is created.
3. Hash the password with bcrypt via Passlib: `pwd_context.hash(password)`.
4. Insert a `User` row with the hashed password and the `one_user_id` returned from step 2.
5. Return a `UserResponse` (excluding the password hash).

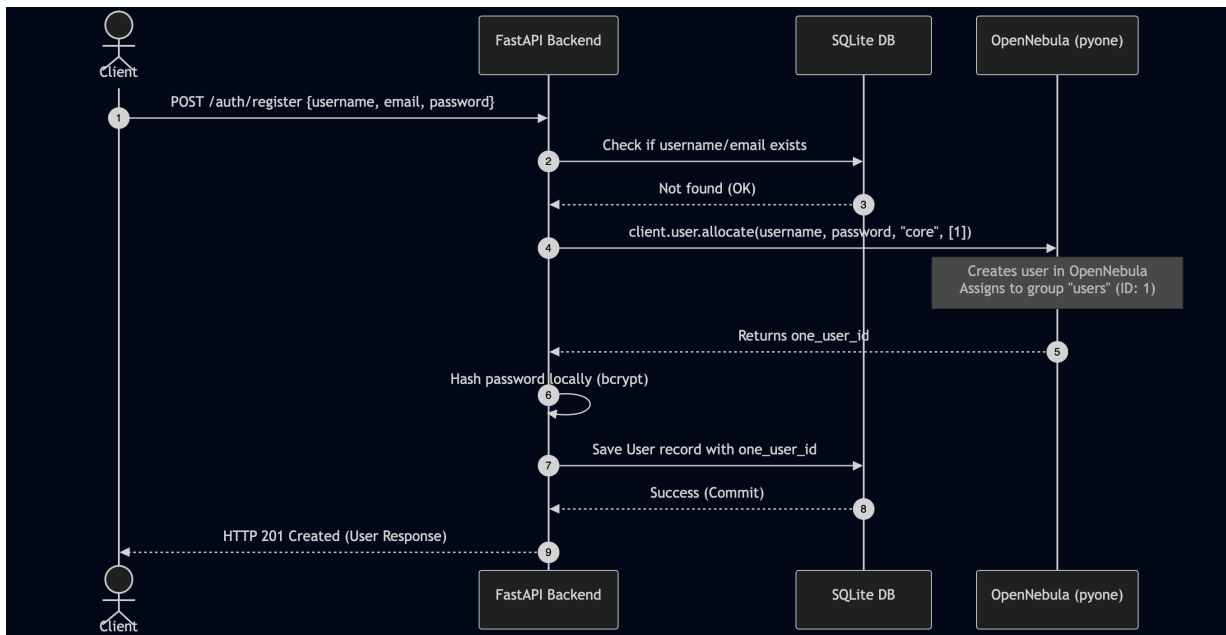


Figure 3: Register user workflow.

This ordering ensures that if the OpenNebula allocation fails, no dangling local record is created. Storage space (MinIO buckets) is not allocated at registration; instead, buckets are provisioned dynamically on-demand during the user's first object storage interaction.

4.4 Login and Token Issuance

The `POST /auth/login` endpoint accepts a `UserLogin` schema (username, password). Authentication is intentionally performed *only* against the local bcrypt hash, not against OpenNebula, for two reasons: first, it avoids a blocking XML-RPC call on the hot authentication path; second, it allows the system to function (for read operations) even if the OpenNebula daemon is temporarily unavailable.

If the bcrypt check passes (`pwd_context.verify(password, user.hash_password)`), the function calls `create_access_token(user.id, user.username)`. The token is signed with the HS256 algorithm using the `SECRET_KEY` environment variable and carries an `exp` claim set to `now + timedelta(hours=24)`. The token is returned in the `TokenResponse` schema as `{"access_token": "...", "token_type": "bearer"}`.

4.5 Request Authentication via JWT Dependency

The `get_current_user` function in `api/auth/jwt.py` is a FastAPI dependency injected into every protected route handler. Its logic is:

```
1 def get_current_user(  
2     token: str = Depends(oauth2_scheme),  
3     db: Session = Depends(get_db)  
4 ) -> User:  
5     payload = jwt.decode(  
6         token, SECRET_KEY, algorithms=["HS256"]  
7     )  
8     user_id = int(payload["sub"])  
9     user = db.query(User).filter(User.id == user_id).first()  
10    if not user or not user.is_active:  
11        raise HTTPException(status_code=401)  
12    user.last_active_at = datetime.now(timezone.utc)  
13    db.commit()  
14    return user
```

Listing 8: JWT dependency with inactivity tracking.

The `last_active_at` update on every authenticated request is the mechanism by which the VM autoscaler tracks user inactivity. Every time a user calls any protected API endpoint, even a read-only `GET /compute/vms`, their inactivity timer resets to zero. Only after 7200 continuous seconds without any API call does the autoscaler consider the user inactive and suspend their VMs.

4.6 Profile Management Endpoints

The auth router exposes additional endpoints for profile management:

Method	Path	Description
POST	<code>/auth/register</code>	Register a new user
POST	<code>/auth/login</code>	Issue a JWT token
GET	<code>/auth/me</code>	Get current user profile + SSH key
PUT	<code>/auth/me</code>	Update email, password, or SSH key
DELETE	<code>/auth/me</code>	Deregister and delete the account

The `GET /auth/me` endpoint returns the `UserResponse` schema augmented with the SSH public key fetched from `get_one_user_ssh_key(current_user.one_user_id)`. This allows the frontend to display the user's current SSH key directly from OpenNebula's authoritative store.

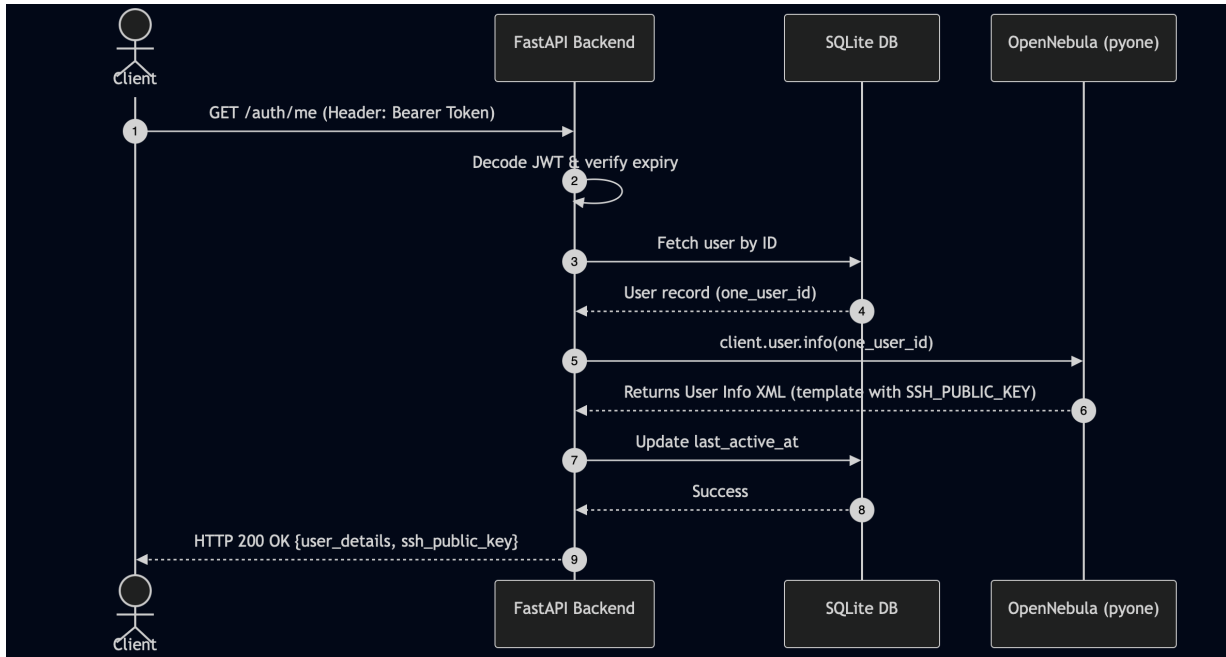


Figure 4: User Retrieval.

The PUT `/auth/me` endpoint accepts a `UserUpdate` schema with optional `email`, `password`, and `ssh_public_key` fields. If `password` is provided, it calls both `pwd_context.hash` locally and `update_one_user_password` in OpenNebula so the two authentication layers remain in sync. If `ssh_public_key` is provided, it calls `update_one_user_ssh_key` to persist the key in OpenNebula’s user template, from which it is subsequently injected into new VMs via the contextualization ISO (`$USER[SSH_PUBLIC_KEY]`).

The DELETE `/auth/me` endpoint calls `delete_one_user(current_user.one_user_id)` first. If that succeeds, it issues a hard delete (`db.delete(current_user)`) and commits. This ordering ensures that if OpenNebula fails (for example, if the user still owns active VM resources), the database record remains intact and the API returns a corresponding error without leaving orphan records.

4.7 Multi-Tenancy Enforcement Summary

Our multi-tenancy model enforces isolation at four independent layers:

1. **JWT bearer token:** Each request must present a valid 24-hour-expiring HS256 token. Without a valid token, the request is rejected at the dependency injection layer before any subsystem logic executes.
2. **SQLite foreign key filters:** Every database query in every router is filtered by `current_user.id`, so queries never return resources owned by other users.
3. **OpenNebula ownership:** VMs and disk images are `chowned` to the user’s `one_user_id` at provisioning time. Even if the API layer were bypassed, the XML-RPC API would reject operations attempted by a different authenticated user.
4. **Label and bucket namespaces:** Containers carry `cloud_user` / `cloud_db_user` labels checked on every operation. Object storage files reside in the `user-<username>` bucket, which is structurally inaccessible from other users’ API sessions.

5 Virtual Machine Management

Virtual machine management is the foundation of our cloud platform. Every other service, containers, databases, object storage, ultimately runs on top of a guest VM allocated and controlled through this subsystem. We imple-

mented the VM layer across four files: the OpenNebula connection singleton (`opennebula/connection.py`), the VM operations library (`opennebula/vm_manager.py`), the block storage library (`opennebula/disk_manager.py`), and the FastAPI compute router (`api/compute/router.py`). Additionally, a WebSocket SSH terminal (`api/compute/terminal.py`), a cluster metrics monitor (`api/compute/monitor.py`), and an SLA policy module (`api/compute/sla.py`) round out the subsystem.

5.1 OpenNebula Connection Layer

The file `opennebula/connection.py` contains the single authoritative source of OpenNebula credentials. It reads `ONE_USER` and `ONE_PASSWORD` from a `.env` file via `python-dotenv` and exposes `get_client()`, which returns:

```
1 pyone.OneServer("http://localhost:2633/RPC2",
2                 session=f"{ONE_USER}:{ONE_PASSWORD}")
```

Listing 9: OpenNebula client factory.

Every module that needs to interact with OpenNebula calls `get_client()` rather than constructing its own `OneServer`, ensuring credentials are managed in one place and the tunnel endpoint is consistent. The XML-RPC port 2633 is accessed via an SSH tunnel established before starting the API server, as the OpenNebula daemon (`oned`) binds only to the loopback interface of the hypervisor host.

5.2 VM Lifecycle Operations

`opennebula/vm_manager.py` wraps the four principal VM lifecycle operations.

Creation (`create_vm`). A VM is instantiated by calling `client.template.instantiate(template_id, name, False, extra_config, False)`. Before this call, `create_vm` builds an `extra_config` string that overrides hardware parameters (CPU, memory, disk size) and injects a `CONTEXT` block. The context block contains:

- `NETWORK = "YES"` and `TOKEN = "YES"` to enable the `one-context` agent.
- `SSH_PUBLIC_KEY = "$USER[SSH_PUBLIC_KEY]"` (substituted with the user's actual key if stored in their OpenNebula template).
- `START_SCRIPT_BASE64`, a Base64-encoded shell script that overrides `/etc/resolv.conf` with Google and Cloudflare DNS resolvers, polls for internet connectivity for up to 30 seconds, then installs and starts the Docker Engine (`apk add docker` on Alpine Linux, `apt-get install -y docker.io` on Ubuntu, each with three retry attempts). This script runs automatically via the `one-context` daemon as the VM boots.

After instantiation, if a `user_id` is provided, `client.vm.chown(one_vm_id, user_id, -1)` transfers ownership so the VM appears in the correct user's resource pool.

State transitions. The function `suspend_vm` issues `client.vm.action("poweroff-hard", one_vm_id)`. We chose `poweroff-hard` rather than `suspend` because the Alpine Linux base image does not include QEMU guest agent or ACPI suspend support; the `suspend` action requires saving full RAM state to disk, which Alpine cannot perform. `resume_vm` issues `client.vm.action("resume", one_vm_id)`, which boots the VM from its persisted disk. `destroy_vm` issues `client.vm.action("terminate-hard", one_vm_id)`, which instructs the hypervisor to destroy the QEMU domain and purge virtual disk files from the datastore.

State and metrics retrieval. `get_vm` calls `client.vm.info(one_vm_id)` and passes the result to the internal `_vm_to_dict` helper, which extracts: the integer `STATE` mapped to a human-readable string via the `VM_STATES` dictionary (e.g., `3 → "ACTIVE"`), the `LCM_STATE` sub-state (where value 3 indicates `RUNNING` with the guest OS fully booted), CPU usage in percent from `MONITORING.CPU`, memory usage in megabytes from

MONITORING.MEMORY (converted from kilobytes), the assigned IP address from `TEMPLATE.NIC[0].IP`, and the root disk size from `TEMPLATE.DISK[0].SIZE`.

`list_all_vms` calls `client.vmpool.info(-2, -1, -1)` to retrieve all VMs visible to the authenticated administrator, then calls `client.vm.info(vm.ID)` individually for each VM to obtain up-to-date monitoring data (the pool endpoint's monitoring fields may be stale). `list_vms_by_one_user` filters the full list by `one_owner_id`.

Image listing. `list_templates` queries `client.templatepool.info(-2, -1, -1)` and returns a list of `{"id": int, "name": str}` dicts for display in the dashboard VM-creation modal, with a hardcoded fallback to `{0: "Alpine Linux 3.20"}` if OpenNebula is unreachable.

SSH user detection. `get_ssh_user_by_template` maps a template ID to the default SSH username (`"root"` for Alpine, `"ubuntu"` for Ubuntu templates). The result is cached in `_TEMPLATE_USER_CACHE` to avoid repeated XML-RPC calls.

5.3 Compute Router Endpoints

The compute router (`api/compute/router.py`) exposes the following endpoints under the `/compute` prefix.

Method	Path	Description
POST	<code>/vms</code>	Provision a new VM
GET	<code>/vms</code>	List the user's active VMs
GET	<code>/vms/{vm_id}</code>	Get live metrics for one VM
DELETE	<code>/vms/{vm_id}</code>	Terminate a VM
POST	<code>/vms/{vm_id}/start</code>	Resume a stopped VM
POST	<code>/vms/{vm_id}/stop</code>	Suspend a running VM
GET	<code>/status</code>	Cluster metrics and SLA config
GET	<code>/vms/{vm_id}/metrics</code>	Historical CPU/memory data points
GET	<code>/energy-stats</code>	Energy savings accounting
GET	<code>/templates</code>	Available OpenNebula templates
POST	<code>/prewarm</code>	Trigger background VM pre-warming

The POST `/vms` handler enforces the `MAX_VMS` SLA limit per user before attempting any OpenNebula call. It first checks whether a pre-warmed standby VM is available: it lists all VMs, filters for names beginning with `prewarmed-vm-`, checks that the state is `ACTIVE` and `LCM_STATE == 3`, and verifies the VM is not already registered as active in `vm_instances`. If a standby is found, it renames it via `client.vm.rename` and transfers ownership via `client.vm.chown`, operations that complete in under one second because the VM is already running. If no standby is available, it falls back to `create_vm`, which takes approximately 85–110 seconds for a cold boot.

The DELETE `/vms/{vm_id}` handler performs cascade cleanup: after terminating the VM in OpenNebula, it attempts to locate any `DBInstance` records whose backing container resided on that VM and deletes them from SQLite, preventing dangling metadata from pointing at containers that no longer exist.

The GET `/compute/energy-stats` endpoint implements the energy accounting formula described in Section 1: it sums the wall-clock uptime of all `VMInstance` records (both active and terminated, using `terminated_at` or the current timestamp), computes the baseline potential hours as $(\text{now} - \text{first_created}) \times \text{MAX_VMS}$, subtracts actual hours to obtain hours saved, and converts using 50 W/VM and 0.4 kg CO₂ /kWh.

The comparative execution path between dynamic standby allocation and cold booting is visually mapped in Figure 5.

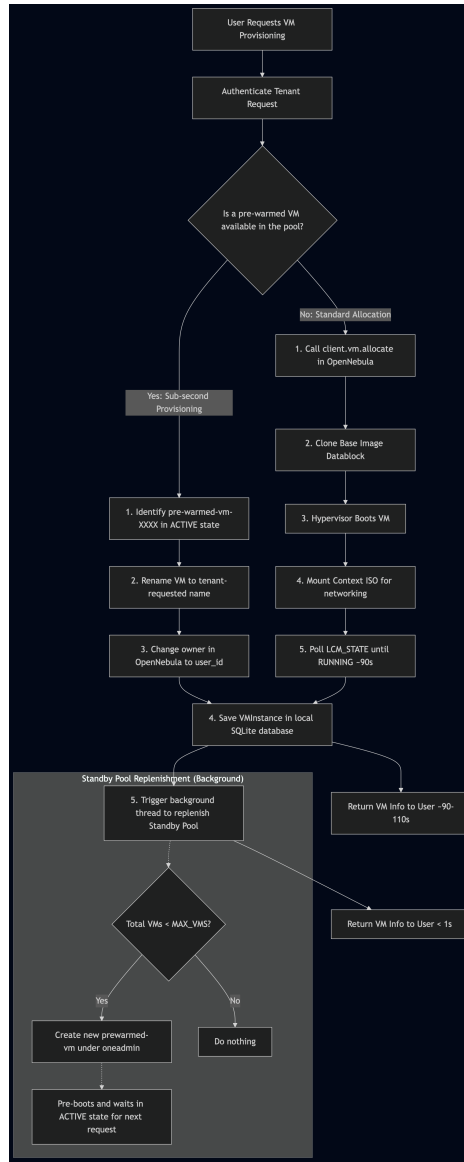


Figure 5: VM Provisioning Lifecycle: Standby pool allocation versus Cold Boot.

5.4 Resource Monitor

`api/compute/monitor.py` exports a single function, `get_cluster_metrics()`, which calls `list_all_vms()`, filters for VMs in `ACTIVE` state with `lcm_state == 3` that are not pre-warmed standbys (names not starting with `prewarmed-vm-`), and computes aggregate `avg_cpu_pct` and `avg_memory_mb`. This function is called by the autoscaler's check loop and by the `GET /compute/status` endpoint.

5.5 SLA Policy Constants

All SLA thresholds are centralised in `api/compute/sla.py`:

Constant	Value
MIN_VMS	0
MAX_VMS	5
SCALE_UP_CPU_PCT	70.0%
SCALE_DOWN_CPU_PCT	20.0%
SCALE_DOWN_WINDOW_SEC	120 seconds
CHECK_INTERVAL_SEC	30 seconds
USER_INACTIVITY_TIMEOUT_SEC	7200 seconds (2 hours)
DEFAULT_TEMPLATE_ID	0 (Alpine Linux 3.20)

MIN_VMS = 0 means the system may scale down to zero VMs when no user is active, enabling full energy savings. The autoscaler only destroys VMs that were created by itself (names prefixed with `autoscale-vm-`), leaving user-provisioned VMs untouched.

5.6 WebSocket SSH Terminal

`api/compute/terminal.py` implements a browser-accessible interactive SSH session using the WebSocket protocol. The router mounts the endpoint at `ws://localhost:8000/terminal/{vm_ip}`.

On connection, the handler uses Paramiko [8] to:

1. Connect to the SSH gateway (PonchaLaptop), configured via the `GATEWAY_IP`, `GATEWAY_PORT`, `GATEWAY_USER`, and `GATEWAY_PASSWORD` environment variables.
2. Open a `direct-tcpip` forwarding channel to port 22 of the target VM IP.
3. Connect a second Paramiko `SSHClient` to the VM through that channel socket, using public key authentication (`~/.ssh/id_rsa` or `~/.ssh/id_ed25519`).
4. Invoke an interactive shell with `invoke_shell(term='xterm', width=80, height=24)`.

Figure 6 illustrates the multi-hop TCP tunnel and bidirectional proxying sequence that enables terminal sessions directly in the browser.

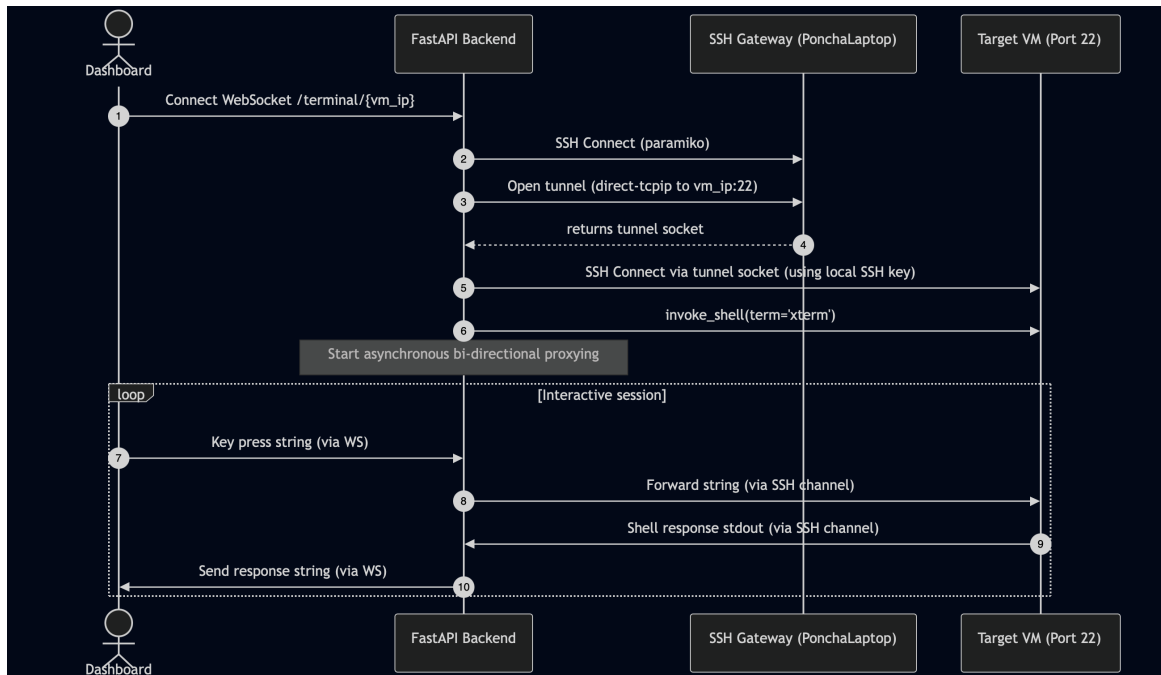


Figure 6: Sequence diagram of the WebSocket SSH terminal tunnel through the gateway.

Two concurrent asyncio coroutines then proxy data bidirectionally: `b_to_ws` reads up to 4096 bytes from the Paramiko channel whenever `channel.recv_ready()` is true and sends the decoded string to the WebSocket; `ws_to_b` awaits keystrokes from the WebSocket and writes them directly to the channel. The VM's IP is looked up in OpenNebula's VM pool to determine whether to use `root` (Alpine) or `ubuntu` as the SSH username.

6 Storage Services

Our platform provides two distinct storage services: an object storage service backed by MinIO [3] for unstructured files such as datasets, model weights, and configuration blobs, and a block storage service backed by OpenNebula DATABLOCK images for raw persistent volumes that can be attached directly to virtual machines. Both services expose REST endpoints under the `/storage` prefix and enforce per-user isolation.

6.1 Object Storage: MinIO Integration

MinIO is an open-source, S3-compatible object storage server. We deploy it as a standalone process on the hypervisor host, listening on port 9002 for data-plane operations and port 9003 for its web console. Because MinIO implements the S3 API in its entirety, the standard `minio` Python client library can communicate with it using the same method signatures as Amazon S3.

Client configuration. `api/storage/minio_client.py` creates a module-level `Minio` client instance pointed at the configured endpoint (default: `localhost:9002`). The access key and secret key are read from the `MINIO_ACCESS` and `MINIO_SECRET` environment variables. The client is initialised with `secure=False` because the connection is purely local (or over the same trusted LAN segment as the API server); adding TLS would require certificate management that is out of scope for the current deployment.

Per-user bucket isolation. The central design decision for object storage isolation is the *per-user bucket*. The helper function `bucket_for(username)` returns the string `f"user-{username}"`, producing a globally unique bucket name such as `user-alice`. The function `ensure_bucket(username)` checks whether this bucket already exists using `client.bucket_exists(bucket_name)` and calls `client.make_bucket(bucket_name)` only if it does not. `ensure_bucket` is called at the start of every object operation, making bucket creation fully transparent: the first upload automatically provisions the bucket without requiring a separate registration step.

This model provides tenant isolation at the namespace level of MinIO itself. Even if an attacker obtained the MinIO root credentials, there is no mechanism for one user's bucket to reference another user's objects: the API layer enforces the `bucket_for(current_user.username)` mapping and never accepts a caller-supplied bucket name.

6.2 Object Lifecycle Operations

`api/storage/minio_client.py` implements four core object operations:

`upload_file(username, file_name, data, length, content_type)`. Calls `ensure_bucket`, then delegates to `client.put_object(bucket_name, file_name, data, length, content_type=content_type)`. The `data` parameter is a binary file-like object obtained from FastAPI's `UploadFile.file`, and `length` is the declared content-length header. If the client omits the content-length (as many HTTP clients do for streaming uploads), the caller passes `-1` and sets `part_size=10MB` to enable MinIO's multipart upload path, which buffers internally and does not require a pre-declared size.

`list_files(username)`. Calls `ensure_bucket`, then calls `client.list_objects(bucket_name)`. The result is a generator of `Object` records; the function materialises them into a list of dicts containing `object_name`, `size` (bytes), and `last_modified` (ISO 8601 string). Only objects within the user's own bucket are returned.

`download_file(username, file_name)`. Calls `client.get_object(bucket_name, file_name)` and returns the raw HTTP response object, which exposes a `read()` method suitable for streaming to the FastAPI caller via a `StreamingResponse`.

`delete_file(username, file_name)`. Calls `client.remove_object(bucket_name, file_name)`.

Figure 7 illustrates the sequence of operations during a file upload, highlighting the transparent auto-provisioning of the user's isolated S3 bucket.

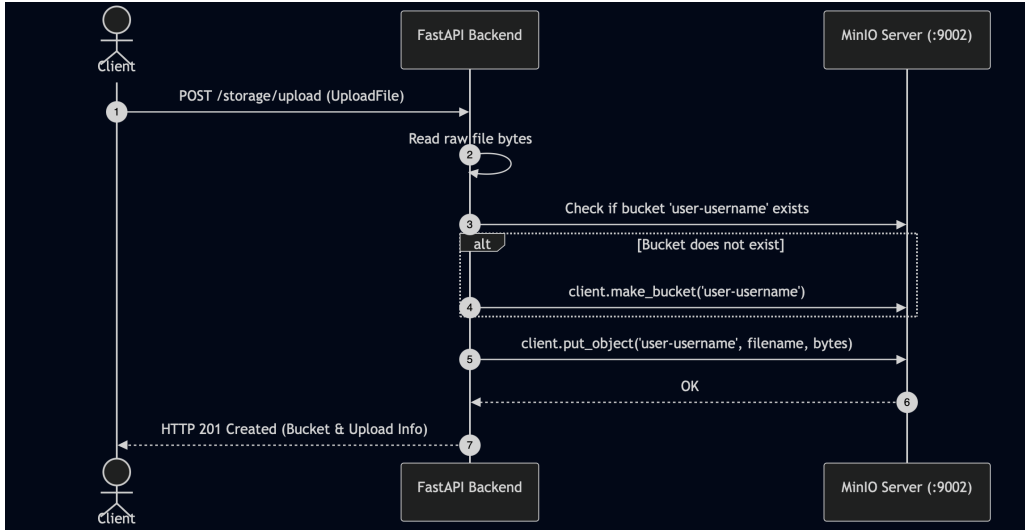


Figure 7: Sequence diagram of the file upload and dynamic bucket auto-provisioning flow.

6.3 Object Storage API Endpoints

The storage router (`api/storage/router.py`) exposes the following object storage endpoints under `/storage`:

Method	Path	Description
POST	<code>/storage/upload</code>	Upload a file (multipart/form-data)
GET	<code>/storage/files</code>	List files in the user's bucket
GET	<code>/storage/download/{name}</code>	Download a file
DELETE	<code>/storage/files/{name}</code>	Delete a file

The upload endpoint accepts an `UploadFile` parameter (FastAPI's multipart wrapper). It reads `file.filename` as the object key, inspects the `Content-Type` header, and passes `file.file` (a `SpooledTemporaryFile`) as the data stream. The download endpoint streams the MinIO response body directly to the client with `media_type="application/octet-stream"` and a `Content-Disposition: attachment` header derived from the requested filename.

6.4 Block Storage: OpenNebula Datablock Images

Block storage serves a different purpose from object storage: it provides a raw block device that can be formatted with any filesystem and mounted inside a running VM, enabling use cases such as persistent databases, large datasets, or application state that must survive container restarts.

The implementation resides in `opennebula/disk_manager.py` and is exposed through the same `/storage` router under the `/storage/disks` sub-path.

Disk creation. The POST `/storage/disks` endpoint accepts a `DiskCreate` schema with `name` and `size_gb` fields. It calls `create_disk(name, size_gb, current_user.one_user_id)`, which allocates an image of type `DATABLOCK` in datastore 1:

```

1  template = f'''
2  NAME = "{name}"
3  TYPE = DATABLOCK
4  SIZE = {size_gb * 1024}
5  FORMAT = raw
6  DRIVER = qcow2
7  '''
8  one_image_id = client.image.allocate(template, 1)
9  client.image.chown(one_image_id, one_user_id, -1)

```

Listing 10: OpenNebula DATABLOCK allocation.

The image is recorded in the local `disk_instances` SQLite table with its `one_image_id`, `name`, `size_gb`, and `user_id`. Because the image's initial state is `LOCKED` while the empty backing file is written to the datastore, the endpoint returns immediately with status 201 `Created` and the caller can poll GET `/storage/disks/{id}` to observe the state change to `READY`.

Disk attach and detach. The POST `/storage/disks/{id}/attach` endpoint accepts a `vm_id` (local SQLite primary key), resolves the corresponding `one_vm_id`, and calls `attach_disk(one_vm_id, one_image_id)`:

```

1  client.vm.attach(one_vm_id, f'DISK=[IMAGE_ID="{one_image_id}"]')

```

Listing 11: Hot-plug disk attachment.

OpenNebula translates this XML-RPC call into a `virsh attach-disk` command issued to the host's `libvirt` daemon, which presents the QEMU virtual disk as a new `/dev/vd*` device to the running guest kernel. No guest reboot is required. The image state transitions from `READY` to `USED` once the attachment completes.

The POST `/storage/disks/{id}/detach` endpoint calls `get_attached_vm_and_disk_index` to discover which VM currently holds the disk (by scanning all the user's VMs) and the disk's index within that VM's `TEMPLATE.DISK` list, then calls `client.vm.detach(one_vm_id, disk_index)`, which reverses the hot-plug. The image returns to `READY`.

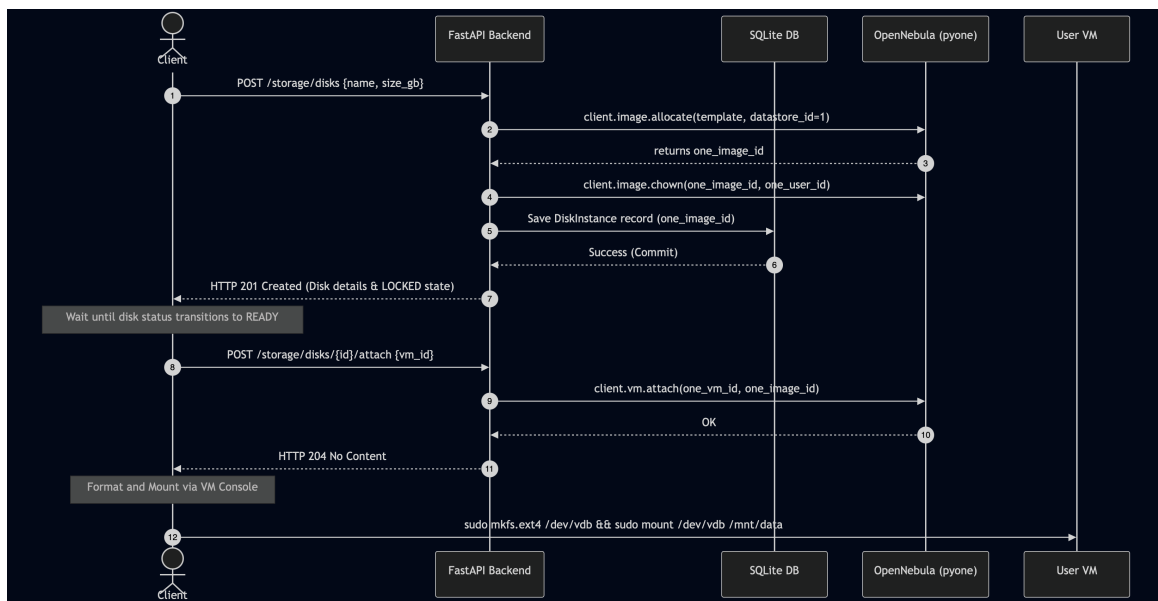


Figure 8: Sequence diagram of block storage allocation, hot-plug attachment, and guest formatting.

Disk listing and deletion. GET /storage/disks queries the `disk_instances` table by `user_id` and enriches each record with the current OpenNebula image state retrieved via `client.image.info`. DELETE /storage/disks/{id} calls `client.image.delete(one_image_id)` to permanently remove the backing file from the datastore and then deletes the `DiskInstance` row from SQLite.

6.5 Block Storage API Endpoints

Method	Path	Description
POST	/storage/disks	Allocate a new block device
GET	/storage/disks	List the user's block devices
DELETE	/storage/disks/{id}	Delete a block device
POST	/storage/disks/{id}/attach	Hot-plug attach to a running VM
POST	/storage/disks/{id}/detach	Hot-plug detach from a running VM

The two storage subsystems are deliberately decoupled. Object storage is stateless from the VM perspective (files are accessed over the S3 HTTP API rather than appearing as block devices), whereas block storage is tightly coupled to a specific VM's device tree. A user may therefore use both services simultaneously: for example, uploading training data to their MinIO bucket and persisting model checkpoints to a block volume mounted at /data inside their VM.

7 Container Management Service

Our container service allows authenticated users to launch, list, inspect, start, stop, and remove Docker containers that run directly on their provisioned OpenNebula virtual machines. Rather than deploying containers on a centralised cluster, each user's VMs act as Docker host nodes. The FastAPI backend communicates with those Docker daemons over SSH, so no additional network exposure is required on the VM: only port 22 needs to be reachable through the gateway.

7.1 Docker SDK SSH Integration

The Docker SDK for Python [9] supports connecting to a remote daemon by setting `base_url` to an `ssh://` URI and passing `use_ssh_client=True`. The connection string format is:

```
1 docker.DockerClient(  
2     base_url=f"ssh://{ssh_user}@{ip}",  
3     use_ssh_client=True  
4 )
```

This causes the Docker SDK to invoke the system's `ssh` binary (or Paramiko under the hood) to forward the Docker Unix socket over the SSH connection, after which all standard Docker SDK operations, image management, container creation, inspection, log retrieval, and statistics, work transparently against the remote daemon.

Because the VMs reside on a private `172.16.100.*` network behind the SSH gateway, the Docker SDK's SSH client must route through the same gateway used by the WebSocket terminal. The user's local `~/.ssh/config` (or the system SSH agent) must include the gateway as a `ProxyJump` host. Alternatively, the `self_heal_docker` fallback uses Paramiko to open a `direct-tcpip` channel explicitly.

7.2 User Isolation via Docker Labels

Every container launched through our API is stamped with the Docker label `cloud_user=<username>` at creation time:

```

1  container = client.containers.create(
2      image,
3      name=f"{username}-{name}",
4      labels={"cloud_user": username},
5      ...
6  )

```

All list, inspect, start, stop, and remove operations verify this label before proceeding. The `list_containers` function passes `filters={"label": f"cloud_user=username"}` to `client.containers.list(all=True)`, so only the calling user's containers are ever returned. Point-lookup functions (`get_container`, `start_container`, `stop_container`, `remove_container`) retrieve the container by its short ID and then assert `container.labels.get("cloud_user")==username`; if the assertion fails, a `PermissionError` (HTTP 403) is raised. Container names are prefixed with the username (`<username>-<name>`), so naming collisions between tenants are structurally impossible.

7.3 Dynamic Port Assignment

Host port bindings are declared as `ports={container_port: None}` in the `containers.create` call. Setting the host side to `None` instructs Docker to assign a random free port from the ephemeral range. This policy prevents port conflicts between multiple containers belonging to different users that both wish to expose the same container port (e.g., both exposing `80/tcp`). After the container starts, the actual assigned port is retrieved from `container.ports`, which maps container ports to lists of `HostIp/HostPort` dicts. The caller receives this mapping in the `ContainerResponse` schema so they know which host port to target.

7.4 Container Lifecycle Operations

`api/containers/docker_client.py` implements the full container lifecycle.

launch_container. The function first calls `ensure_user_has_running_vm` (described in Section 5) to guarantee that at least one running VM with Docker available is assigned to the user. It then obtains a `DockerClient` for the selected VM, removes any pre-existing stopped container with the same computed name (to avoid `docker: name is already in use` errors on re-deployment), pulls the requested image if not locally cached (`client.images.get` → `client.images.pull` on `ImageNotFound`), calls `containers.create` with the label, environment variables, and port bindings, and then calls `container.start()`. If `start()` raises an exception (e.g., the image has an invalid entrypoint or a port conflict occurs), the container is immediately removed with `container.remove(force=True)` to prevent orphaned, stopped containers from accumulating in the user's namespace.

The step-by-step transaction sequence of VM discovery, load balancing node selection, dynamic port mapping, and automatic error recovery during a container launch is illustrated in Figure 9.

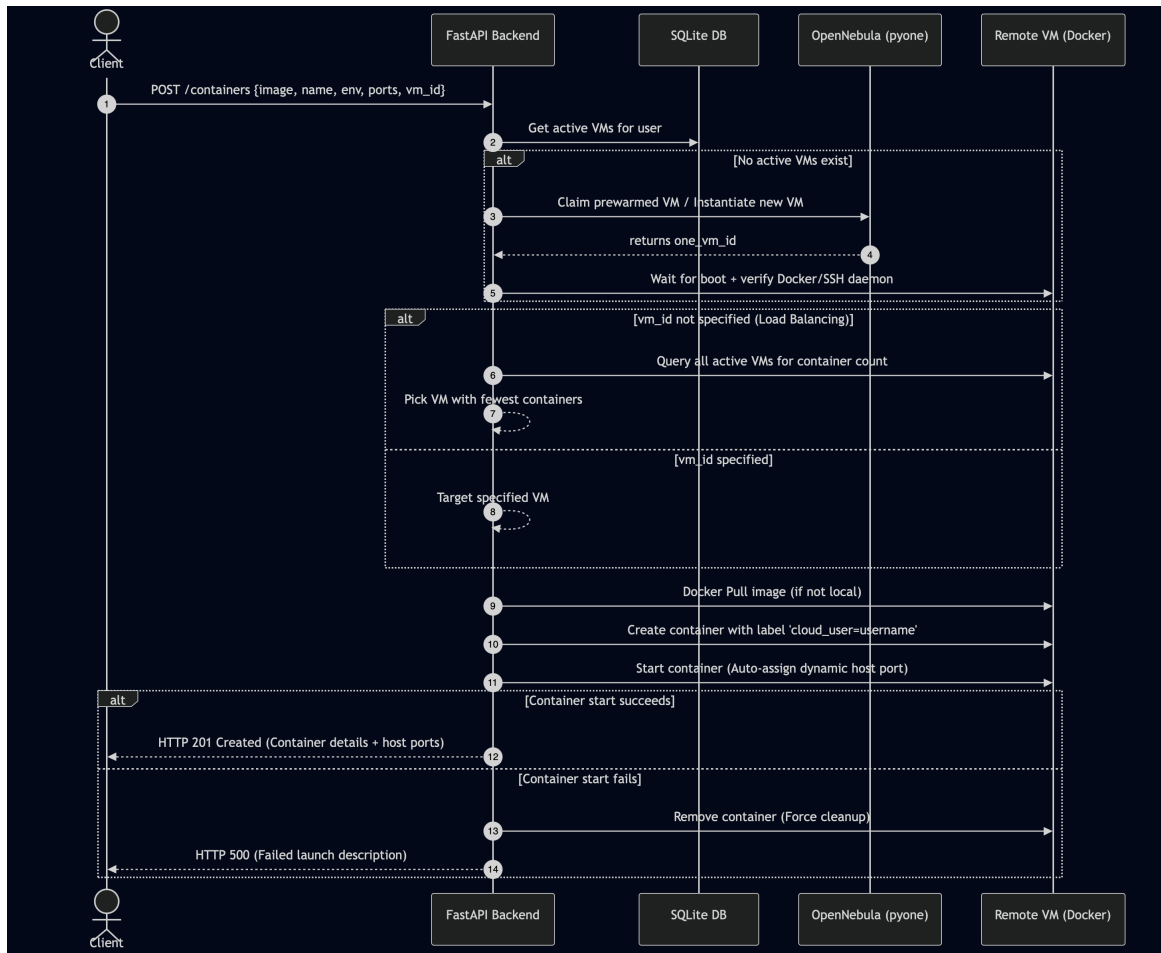


Figure 9: Sequence diagram of the dynamic container placement, launch, and recovery flow.

list_containers. Calls `get_all_clients` to open Docker connections to all the user's active VMs in parallel (via `ThreadPoolExecutor`), then submits a `fetch_containers` task per VM to the executor. Each task calls `client.containers.list(all=True, filters={"label": "cloud_user=username"})` and appends the `vm_id` to each container dict so the dashboard can display which VM hosts each container. Results are merged into a single flat list.

get_container. Also uses `get_all_clients` with `ThreadPoolExecutor`, dispatching a `search_container` task per VM. Each task calls `client.containers.get(container_id)`, verifies the owner label, and returns the container info dict. The executor returns results as a generator; the first non-None result ends the search.

stop_container and start_container. Search VMs sequentially (the container's location is typically known from a prior `get_container` call) and call `container.stop()` or `container.start()` respectively. After starting, a fresh `container.reload()` is performed so the returned port mapping reflects the newly assigned host port.

remove_container. Issues `container.remove(force=True)`, which stops the container if running and then removes it, releasing its host port immediately.

get_container_logs. Returns up to `tail` lines (default 100) of combined stdout/stderr from `container.logs(stdout=True, stderr=True, tail=tail)`.

get_container_stats. Fetches a single snapshot via `container.stats(stream=False)` and computes memory usage percentage as $(\text{usage}/\text{limit}) \times 100$ and CPU percentage using the same delta formula as the container

autoscaler (Section 9).

7.5 Self-Healing Docker Installation

Because VMs are provisioned from a minimal Alpine Linux image, the Docker daemon may occasionally fail to start due to network timing issues during the VM's first boot (e.g., the `apk update` step fails if the DNS resolver is not yet configured). The `self_heal_docker` function in `docker_client.py` provides a remediation path: when a VM reports `LCM_STATE == 3` (guest OS running) but Docker is unreachable, `ensure_user_has_running_vm` triggers `self_heal_docker(ip)`, which connects to the VM via a Paramiko tunnel and checks the Docker daemon status. On Alpine, it runs `service docker status`; if Docker is not started, it runs `service docker start`; if the Docker binary does not exist, it installs it from scratch (`apk update && apk add docker && rc-update add docker default && service docker start`). After self-healing, the boot loop continues polling until Docker responds to `client.ping()`.

7.6 Container API Endpoints

The container router (`api/containers/router.py`) exposes the following endpoints under the `/containers` prefix:

Method	Path	Description
POST	<code>/containers</code>	Launch a new container
GET	<code>/containers</code>	List the user's containers
GET	<code>/containers/{id}</code>	Inspect a single container
POST	<code>/containers/{id}/start</code>	Start a stopped container
POST	<code>/containers/{id}/stop</code>	Stop a running container
DELETE	<code>/containers/{id}</code>	Remove a container
GET	<code>/containers/{id}/logs</code>	Retrieve stdout/stderr logs
GET	<code>/containers/{id}/stats</code>	Live CPU and memory metrics

The request schema for container creation is `ContainerCreate` (defined in `api/containers/schemas.py`), which accepts `image` (Docker image name), `name` (user-supplied suffix), optional `env` (key-value environment variables), and optional `ports` (list of container port strings such as `"80/tcp"`). The response schema `ContainerResponse` returns `container_id` (Docker short ID), `name`, `image`, `status`, `ports` (the full Docker port mapping dict), and `created`.

8 Database-as-a-Service

Our Database-as-a-Service (DBaaS) layer allows authenticated users to provision, inspect, connect to, restart, and deprovision `postgres:16-alpine` database instances without any manual PostgreSQL administration. Provisioned databases run as Docker containers on the user's VMs, with data persisted to a host directory path (`/var/lib/postgresql/data-{instance_name}`) on the VM's filesystem so that it survives container restarts. Users receive a complete DSN (Data Source Name) on creation and can optionally extend their deployment into a replicated cluster fronted by an HAProxy TCP load balancer (described in Section 9).

8.1 Database Provisioning

The core provisioning logic is implemented in `api/database_service/db_manager.py` in the function `provision_db`. Its steps are as follows.

Secret generation. A cryptographically random 24-character alphanumeric password is generated using Python’s `secrets` module:

```
1 password = ''.join(  
2     secrets.choice(string.ascii_letters + string.digits)  
3     for _ in range(24)  
4 )
```

Listing 12: Random password generation for database isolation.

This password is stored in the `db_instances` SQLite table alongside the database name and never returned after the initial creation response, enforcing a “show once” credential model. The `POSTGRES_PASSWORD` environment variable passed to the container uses this password; the database administrator user (set to the tenant’s username) authenticates with it.

Volume and container creation. The function resolves a Docker client for one of the user’s running VMs by calling `ensure_user_has_running_vm`. It then creates the container with a host directory mount for data persistence:

```
1 container = client.containers.run(  
2     "postgres:16-alpine",  
3     name=f"db-{username}-{instance_name}",  
4     detach=True,  
5     environment={  
6         "POSTGRES_USER": username,  
7         "POSTGRES_PASSWORD": password,  
8         "POSTGRES_DB": db_name,  
9     },  
10    volumes={  
11        f"/var/lib/postgresql/data-{instance_name}": {  
12            "bind": "/var/lib/postgresql/data",  
13            "mode": "rw"  
14        }  
15    },  
16    ports={"5432/tcp": None},  
17    labels={"cloud_db_user": username}  
18 )
```

Listing 13: PostgreSQL container provisioning.

Declaring the host port as `None` causes Docker to assign a random free host port from the ephemeral range. The `cloud_db_user=<username>` label serves the same isolation function as `cloud_user` for containers: it restricts which databases appear in a user’s listing and prevents cross-tenant access.

Port polling. After starting the container, `provision_db` enters a polling loop: it calls `container.reload()` repeatedly, inspecting `container.ports["5432/tcp"][0]["HostPort"]` until a non-null value appears (Docker’s port mapping is populated asynchronously after the container starts), sleeping one second between attempts for up to 60 retries.

DSN construction. On successful retrieval of the port mapping, the function constructs the connection DSN:

```
1 dsn = (  
2     f"postgresql://{username}:{password}"  
3     f"@{vm_ip}:{host_port}/{db_name}"  
4 )
```

This DSN is returned in the `DBInstanceResponse` schema and is also stored in the `db_instances` table. Users can connect to this endpoint from any machine that can reach the VM's public IP.

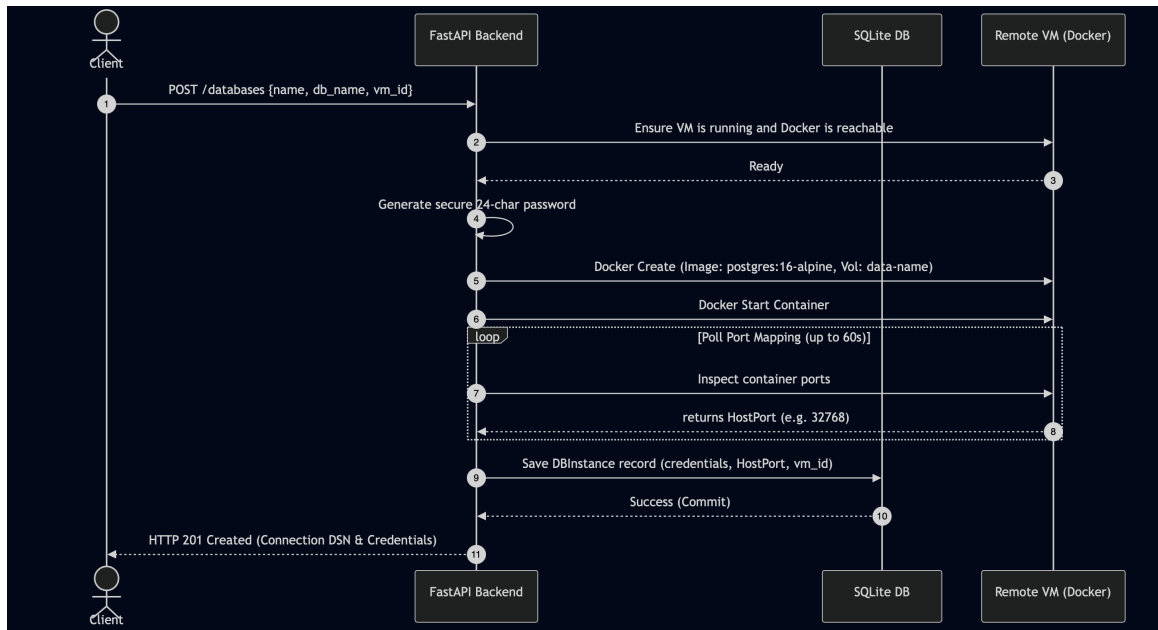


Figure 10: Sequence diagram of the Provision of DBs.

8.2 Container Discovery Across VMs

Because a user may have multiple VMs, and a database container could reside on any of them, operations that target an existing database (inspect, restart, deprovision, metrics) must first locate the container. The function `get_db_container_and_client` in `db_manager.py` performs this search in parallel across all of the user's VMs using `ThreadPoolExecutor` and a `threading.Lock`:

```

1 found = {"client": None, "container": None}
2 lock = threading.Lock()
3
4 def search_vm(vm):
5     client = docker.DockerClient(
6         base_url=f"ssh://{ssh_user}@{vm.ip}",
7         use_ssh_client=True
8     )
9     try:
10         c = client.containers.get(container_id)
11         if c.labels.get("cloud_db_user") == username:
12             with lock:
13                 if found["container"] is None:
14                     found["client"] = client
15                     found["container"] = c
16                 return
17         client.close()
18     except docker.errors.NotFound:
19         client.close()
20
21 with ThreadPoolExecutor() as executor:
22     executor.map(search_vm, vms)
  
```

Listing 14: Parallel container search with first-win lock.

The first thread to locate the container acquires the lock, stores the `(client, container)` pair, and returns. All other threads detect `found["container"]` is not `None` under the lock and immediately close their own Docker clients to release the SSH connection, preventing socket leaks.

8.3 Database Metrics

The `GET /databases/{id}/metrics` endpoint calls `get_db_metrics(username, container_id)`, which connects to the VM hosting the container via `run_ssh_command` (tunnelled through the gateway using Paramiko) and runs two SQL commands via `docker exec`:

```
1 docker exec -e PGPASSWORD=alice_pass db-alice-mydb psql -U alice -d mydb -t -c \
2 "SELECT pg_size_pretty(pg_database_size('mydb'));"
3 docker exec -e PGPASSWORD=alice_pass db-alice-mydb psql -U alice -d mydb -t -c \
4 "SELECT count(*) FROM pg_stat_activity WHERE backend_type = 'client backend';"
```

Listing 15: Database metrics collection via SSH exec.

The first command returns the total on-disk database size (including relation files, indexes, and TOAST tables). The second returns the number of active client connections. Both values are parsed from the `psql` tabular output and returned in the `DBMetricsResponse` schema alongside the container's memory usage (obtained from `container.stats(stream=False)`).

8.4 Database Lifecycle Operations

restart_db. Calls `container.restart()`, which sends `SIGTERM` to the PostgreSQL process, waits for a graceful shutdown (up to 10 seconds), then sends `SIGKILL` if needed. Because the data directory is persisted on the VM filesystem, the database contents are preserved across restarts.

deprovision_db. Calls `container.remove(force=True)`, which stops the running container and removes it. The host directory volume (`/var/lib/postgresql/data-{instance_name}`) is optionally removed if the caller sets `delete_data=True`; otherwise it persists and can be reattached if the user re-provisions a database with the same name. The `DBInstance` row is deleted from SQLite.

8.5 Database API Endpoints

The database router (`api/database_service/router.py`) exposes the following endpoints under `/databases`:

Method	Path	Description
POST	<code>/databases</code>	Provision a new PostgreSQL instance
GET	<code>/databases</code>	List the user's database instances
GET	<code>/databases/{id}</code>	Inspect a single instance
GET	<code>/databases/{id}/metrics</code>	Size, connection count, memory
POST	<code>/databases/{id}/restart</code>	Restart the database container
DELETE	<code>/databases/{id}</code>	Deprovision and optionally delete data

8.6 Clustered PostgreSQL with Streaming Replication

Beyond single-instance DBaaS, our platform supports provisioning a multi-node PostgreSQL cluster through `api/loadbalancer/db_lb.py` (covered in detail in Section 9). The cluster model consists of one write-primary and one or more streaming replicas, with HAProxy providing dual-port query splitting: writes go to port 5432 (primary only) and reads are distributed across all nodes in round-robin via port 5433.

The `POST /loadbalancer/databases/cluster` endpoint accepts a `DBClusterProvisionRequest` containing the cluster name and the desired number of replicas. The orchestration calls `provision_db` once for the primary,

configures WAL-based replication via `pg_hba.conf` modification and `pg_reload_conf()`, runs `pg_basebackup` on each replica to produce a consistent snapshot of the primary's data directory, starts each replica with `standby.signal` in its data directory (causing it to boot in hot-standby mode), and finally deploys an HAProxy container configured with the dual-port frontend/backend architecture.

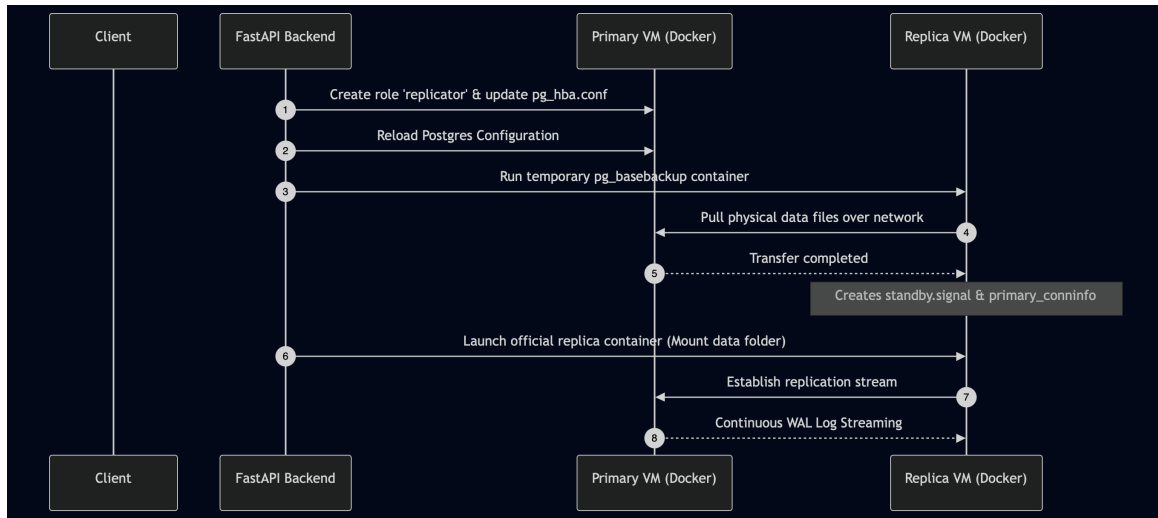


Figure 11: Sequence diagram of the replication of the database.

This design means that the `provision_db` function is reused verbatim for both standalone and clustered deployments: the cluster orchestration layer simply calls it once per node and then applies replication configuration on top. Users connecting to the cluster use the same DSN format (`postgresql://...`) but target the HAProxy VM's IP and port 5432 for writes or port 5433 for reads.

9 Autoscaling and Load Balancing

Elastic scaling is one of the defining properties of a cloud platform. We implemented two independent autoscalers: a *VM-level autoscaler* that controls the number of guest virtual machines in response to aggregate CPU pressure and user inactivity, and a *container-level autoscaler* that adjusts the replica count of named container scale groups. Both autoscalers run as daemon threads managed by FastAPI's lifespan context and operate concurrently with the request-serving event loop. For traffic distribution we implemented two load balancers: an Nginx-based HTTP reverse proxy for container groups and an HAProxy [4] Layer 4 TCP proxy for PostgreSQL clusters.

9.1 VM-Level Autoscaler

The `AutoScaler` class in `api/compute/autoscaler.py` is a `threading.Thread` subclass that runs a `_loop` method, which calls `self._stop_event.wait(timeout=sla.CHECK_INTERVAL_SEC)` in a loop. The stop event is a `threading.Event` set by `autoscaler.stop()` during FastAPI shutdown. A singleton instance named `autoscaler` is imported by `api/main.py` and started in the lifespan context.

Check-and-scale cycle. Every 30 seconds the autoscaler calls `get_cluster_metrics()` to obtain the current `active_vms` count and `avg_cpu_pct` across all non-prewarmed active VMs, then executes the following steps in order:

1. **Metric recording.** For each active VM whose `one_vm_id` matches a `VMInstance` record, a new `VMMetric` row is inserted into SQLite. This provides up to 50 historical data points per VM for the dashboard's CPU/memory charts.

2. **Pre-warming pool maintenance.** If fewer than one standby VM exists (`len(prewarmed_vms) < 1`) and the total VM count is below `MAX_VMS`, a new VM is created under system ownership (no `user_id`), named `prewarmed-vm-<timestamp>`. The next time any user provisions a VM or the autoscaler triggers a scale-up, this standby is claimed rather than a cold boot being initiated.
3. **Inactivity management.** For each user in the local database, the autoscaler computes `inactive_duration = (now - user.last_active_at).total_seconds()`. If this exceeds `USER_INACTIVITY_TIMEOUT_SEC` (7200 s), all of the user's `ACTIVE` non-autoscale VMs are suspended via `suspend_vm(one_vm_id)` and the corresponding `VMInstance.suspended_by_system` flag is set to `True`. Conversely, if the user has recently become active and a suspended VM carries `suspended_by_system = True`, `resume_vm` is called to restore the VM without requiring user intervention.
4. **Scale-up decision.** If `avg_cpu > SCALE_UP_CPU_PCT` and `total < MAX_VMS`, the autoscaler identifies the user whose VMs have the highest average CPU utilisation. It then searches for an available pre-warmed standby VM. If found, it calls `client.vm.rename(one_vm_id, autoscale_name)` followed by `client.vm.chown(one_vm_id, target_user_id, -1)`, which transfers the running VM to the user in under one second. If no standby is available, it falls back to `create_vm`, which takes the full 85–110 second cold-boot cycle. A 120-second cooldown (`_last_scale_up`) prevents consecutive scale-up events before the new VM's metrics stabilise.
5. **Scale-down decision.** If `avg_cpu < SCALE_DOWN_CPU_PCT` and `total > MIN_VMS`, the autoscaler considers only VMs whose names begin with `autoscale-vm-` and whose own CPU is below the threshold. For each candidate, it first performs a *drain check*: it opens a Docker connection `docker.DockerClient(base_url="ssh://<ssh_user>@<ip>")` and lists all containers. If any container is present, the VM is excluded and its idle timer is reset. Only if no containers are found and the VM has been continuously idle for `SCALE_DOWN_WINDOW_SEC` (120 s) does the autoscaler call `destroy_vm` and set `VMInstance.terminated_at`. The idle timer is tracked in the module-level dictionary `_idle_since`, keyed by `one_vm_id`. This two-minute stabilisation window prevents “flapping” where a brief load spike causes a VM to be provisioned and immediately destroyed.

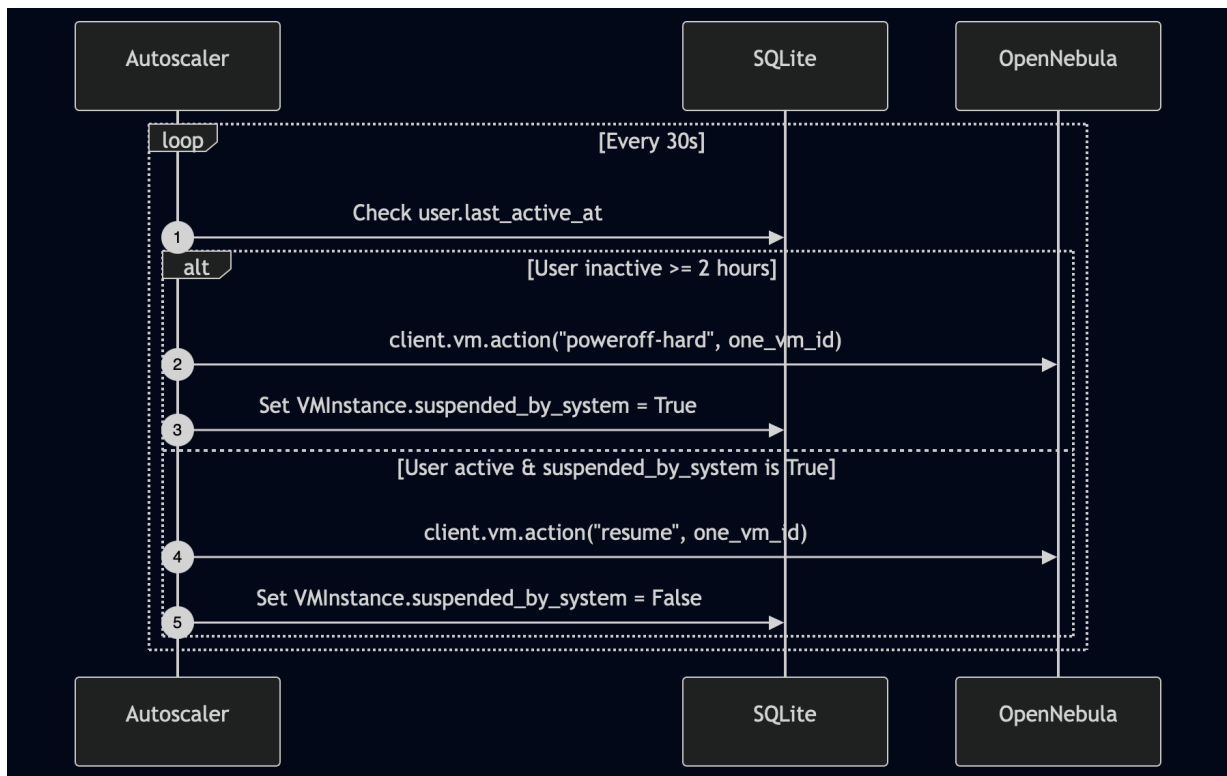


Figure 12: Inactivity Suspend & Resume Flow.

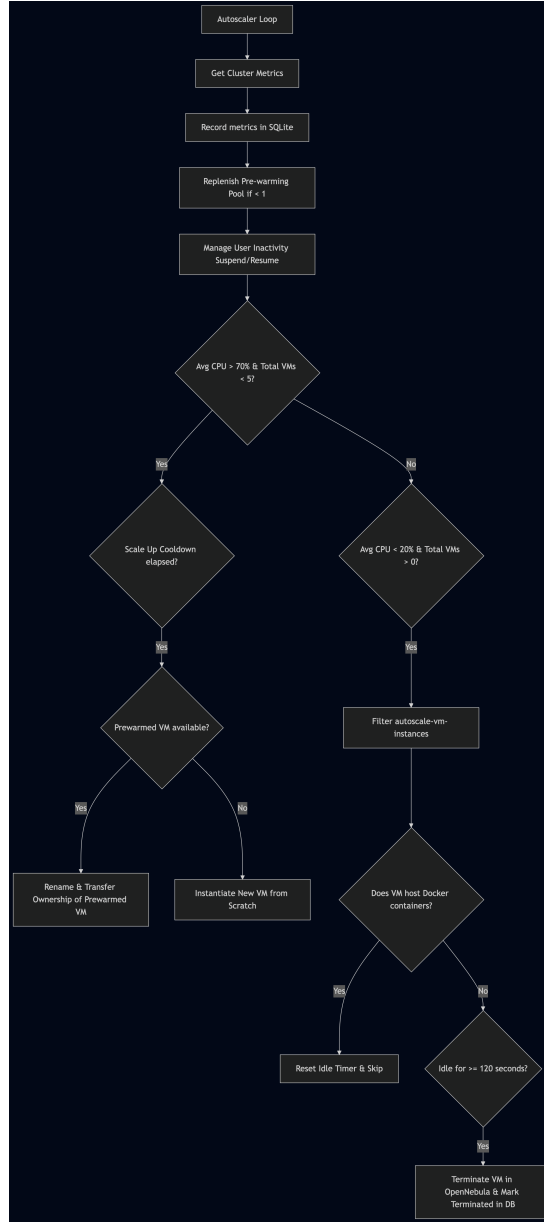


Figure 13: Scale UP & Down Workload Flow.

Post-resume service recovery. When a VM is resumed (either due to user activity detection, manual API requests, or container deployment triggers), the autoscaler queues its `one_vm_id` in a thread-safe dictionary `_recently_resumed`. Once the VM reaches the `ACTIVE` state and its lifecycle state stabilises to running (`LCM_STATE = 3`), the autoscaler runs an automatic service recovery cycle:

- Container Restoration:** It executes an SSH command to list stopped containers (`status=exited` with exit code 255 representing host reboot) and starts them using `docker start`.
- Dynamic Port Re-registration:** Because Docker assigns dynamic host ports upon restart, it queries the new port bindings (`docker port`) and updates the corresponding rows in the SQLite `db_instances` table.
- Load Balancer Reconfiguration:** It rebuilds the custom `haproxy.cfg` configuration for any database clusters on that VM with the new ports and signals HAProxy via `SIGHUP` to reload.
- WAL Replication Correction:** For Standby replica nodes, it runs `ALTER SYSTEM SET primary_conninfo` with the primary's new port and executes `SELECT pg_reload_conf()` to restore streaming replication.

9.2 Container-Level Autoscaler

The `ContainerAutoScaler` class in `api/loadbalancer/container_autoscaler.py` follows the same threading pattern as the VM autoscaler but operates on a 10-second check cycle. Its job is to adjust the replica count of any named container scale group discovered on the user's active VMs.

Scale group discovery. In each cycle, the autoscaler calls `get_all_clients(username)` for every user in the database, which returns a list of `(vm_instance_id, DockerClient)` tuples opened in parallel via `ThreadPoolExecutor`. For each Docker client it lists all containers and inspects their labels. Any container carrying both `scale_group=<name>` and `role=worker` is registered in a `groups` dictionary keyed by group name. Containers with the label `no_autoscale=true` are discovered but silently skipped.

Stabilisation window. When a scale group is seen for the first time, its timestamp is recorded in `self._first_seen`. The group is skipped for the next 60 seconds (`STABILIZATION_SEC`). This prevents premature scale-down immediately after launch: freshly started containers report 0% CPU before any load has arrived, which without stabilisation would trigger an immediate removal. This behaviour mirrors the Horizontal Pod Autoscaler stabilisation window in Kubernetes.

CPU measurement. Rather than relying on Docker's single-snapshot `stats(stream=False)`, which returns a stale `precpu_stats` baseline and therefore always reports 0% CPU on idle containers, the autoscaler takes *two* explicit snapshots one second apart and computes the delta:

```
1 s1 = container.stats(stream=False)
2 time.sleep(1)
3 s2 = container.stats(stream=False)
4 cpu_delta    = s2["cpu_stats"]["cpu_usage"]["total_usage"] \
5               - s1["cpu_stats"]["cpu_usage"]["total_usage"]
6 system_delta = s2["cpu_stats"]["system_cpu_usage"] \
7               - s1["cpu_stats"]["system_cpu_usage"]
8 cpu_pct = (cpu_delta / system_delta) * online_cpus * 100
```

Listing 16: Two-snapshot CPU measurement.

This guarantees a real measurement interval and yields accurate readings even for containers at very low or bursty load.

Scaling rules. After computing `avg_cpu` across all running workers in the group, the autoscaler applies:

- **Scale up** if `avg_cpu > 70%` and `current_count < MAX_REPLICAS` (4): target count increases by 1.
- **Scale down** if `avg_cpu < 5%` and `current_count > MIN_REPLICAS` (1): target count decreases by 1.
- **Cooldown:** a 30-second window (`self._last_scale_time`) prevents consecutive scaling actions for the same group.

When a scaling action is needed, the autoscaler constructs a `ContainerScaleRequest` and calls `scale_container_group`, which provisions or removes workers and regenerates the Nginx configuration.

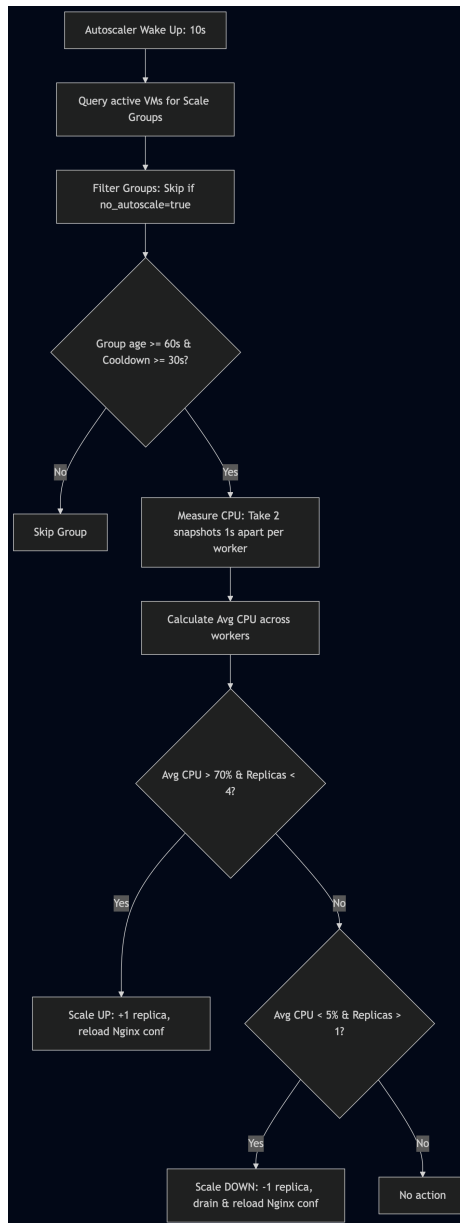


Figure 14: Container Autoscaler Flow.

9.3 Nginx-Based Container Load Balancer

`api/loadbalancer/container_lb.py` implements horizontal scaling of Docker container groups with a dynamically reconfigured Nginx reverse proxy.

Worker placement. New worker containers are launched by calling `ensure_user_has_running_vm`, which selects the least-loaded VM by counting containers on each host in parallel. Port bindings are declared as `ports={container_port: None}`, so Docker assigns a random free host port, preventing conflicts when multiple workers of the same service share a single host.

Nginx configuration generation. After scaling, the load balancer resolves the VM IP and dynamic host port for every active worker. It assembles an `nginx.conf` file containing an `upstream backend_servers {}` block with one `server <ip>:<port>` directive per worker, and a `server { listen 80; }` block that proxies all requests to the upstream pool in round-robin order.

Configuration deployment. The generated `nginx.conf` is written to the load-balancer VM at `/var/lib/nginx-lb-<group\_name>/nginx.conf` via the `write_ssh_file` helper in `api/loadbalancer/ssh_utils.py`. This function establishes an SSH tunnel through the gateway using Paramiko, opens an SFTP client on the resulting transport, and writes the file byte-for-byte. An `nginx:alpine` container is then started (or recreated) with the configuration directory mounted read-only at `/etc/nginx`, exposing port 80 on a randomly assigned host port. The resulting address (`http://<VM_IP>:<LB_PORT>`) is returned to the caller as the single endpoint for the scale group.

Connection pooling. To avoid opening a new Docker SSH connection for every sub-operation within a single scale request, `scale_container_group` maintains a per-call pool dictionary mapping VM IDs to open `DockerClient` instances. All operations within the call reuse the pooled client; the pool is closed in a `finally` block even if the scaling operation fails partway through.

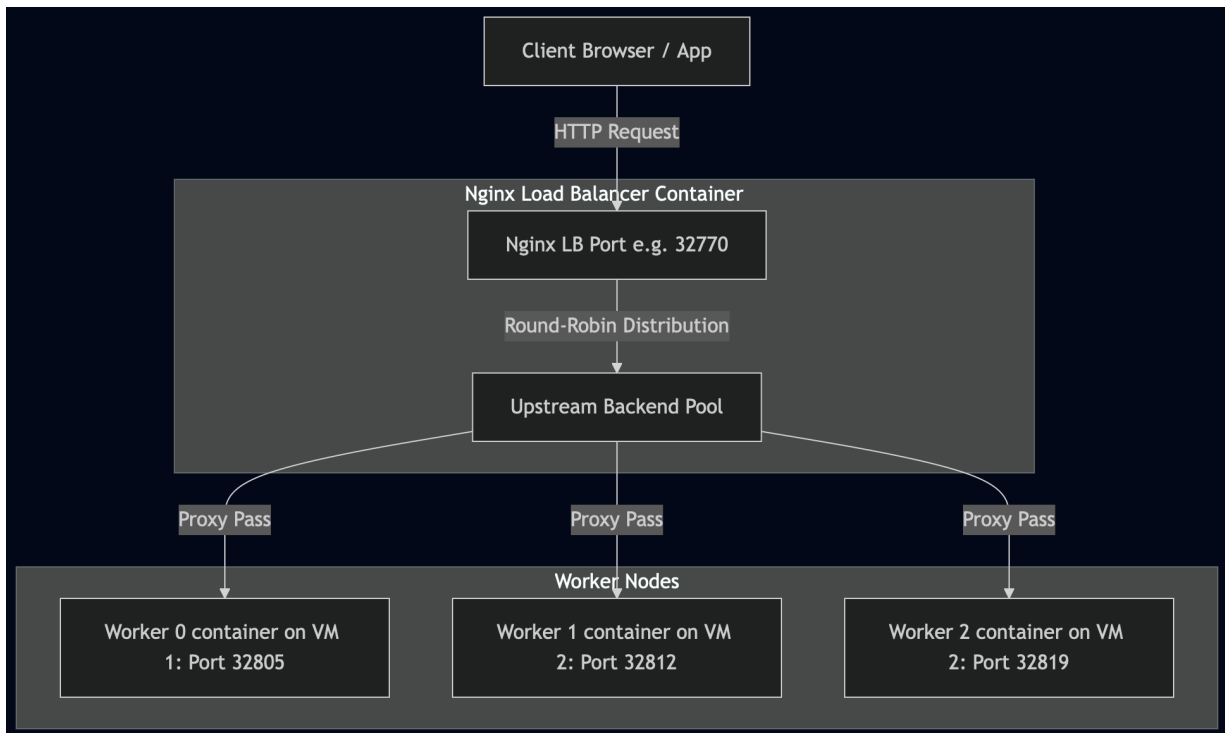


Figure 15: Container loadbalancer Flow.

9.4 HAProxy Database Load Balancer

Database clusters require stronger consistency guarantees than stateless HTTP services. We implemented a PostgreSQL primary/replica cluster with HAProxy as a Layer 4 TCP load balancer, configured with a dual-port strategy to separate write traffic from read traffic.

Cluster provisioning. The `provision_cluster` function in `api/loadbalancer/db_lb.py` orchestrates four sequential steps:

1. **Primary provisioning.** A `postgres:16-alpine` container is launched on the user's first available VM via `db_manager.provision_db`. The container name follows the convention `db-<username>-<cluster_name>-primary`.
2. **Replication configuration.** The system connects to the primary VM via `run_ssh_command` (tunnelled through the gateway) and:
 - Polls `pg_isready` until the primary accepts connections (up to 30 attempts, one per second).

- Executes `CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'replicasecret'` via `docker exec ... psql`.
- Appends host `replication replicator 0.0.0.0/0 scram-sha-256` to `pg_hba.conf`.
- Appends host `all all 0.0.0.0/0 scram-sha-256` to allow HAProxy-routed connections (which arrive at PostgreSQL from the VM bridge IP rather than loopback).
- Calls `SELECT pg_reload_conf()` to apply the changes without restarting.

3. **Replica synchronisation.** For each replica, the system runs a temporary `pg_basebackup` container on the target replica VM:

```

1 docker run --rm -e PGPASSWORD=replicasecret \
2   -v /var/lib/postgresql/data-<cluster>-replica-<i>:/backup \
3   postgres:16-alpine \
4   pg_basebackup -h <primary_ip> -p <port> \
5   -D /backup -U replicator --checkpoint=fast -v -P -R

```

Listing 17: `pg_basebackup` command for replica initialisation.

The `-R` flag causes `pg_basebackup` to create `standby.signal` and write a `primary_conninfo` entry in the data directory. When the replica PostgreSQL container is subsequently started against this data directory, it boots in read-only streaming-replication mode, opening a persistent WAL log connection to the primary.

4. **HAProxy deployment.** An `haproxy:2.8-alpine` container is deployed on a separate VM. Its configuration, generated by `deploy_haproxy_lb` and written to `/var/lib/haproxy-<cluster_name>/haproxy.cfg` via SFTP, defines two frontends:

- **Write frontend** (bind `*:5432`): maps to a single `postgres_primary` backend containing only the primary container's IP and dynamic host port. All `INSERT`, `UPDATE`, `DELETE`, and `DDL` statements must be directed here.
- **Read frontend** (bind `*:5433`): maps to `postgres_replicas`, a `balance roundrobin` backend containing both the primary and all replica instances. Read-only `SELECT` queries are distributed across all nodes.

Both backends are configured with option `tcp-check` so HAProxy performs active TCP health checks; unreachable replicas are automatically marked `DOWN` and removed from the rotation.

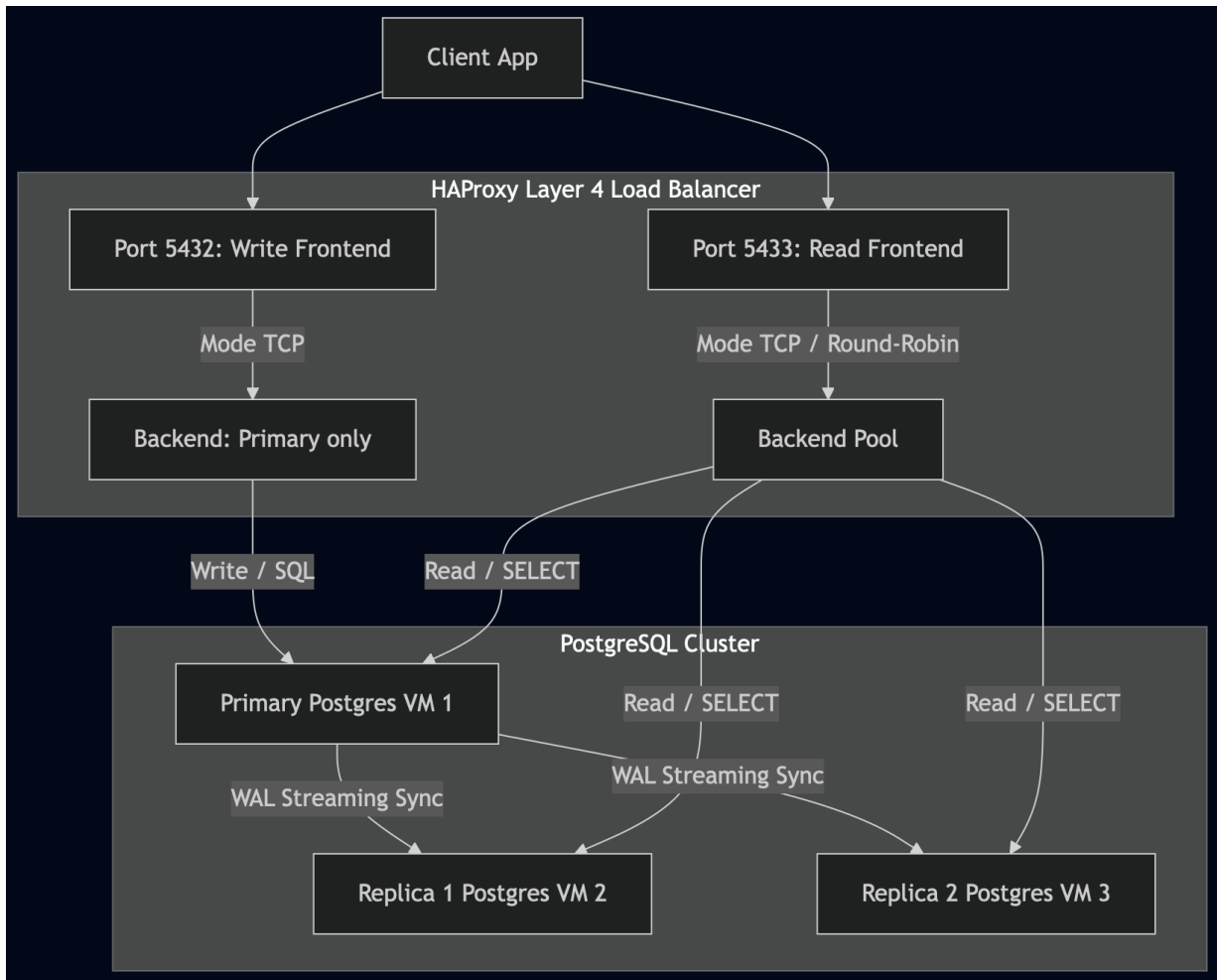


Figure 16: database loadbalancer Flow.

Memory constraints. The Alpine Linux VMs in our environment have only 2048 MB of RAM, shared among the PostgreSQL process, the Docker daemon, and any other containers. HAProxy’s default multi-threaded configuration consumes approximately 57 MB. We constrain it with `nbthread 1` (single worker thread) and `maxconn 100` globally and `maxconn 50` in the defaults block, reducing RSS to approximately 15 MB.

Hot reload. When the cluster is scaled (via `POST /loadbalancer/databases/cluster/{name}/scale?replicas=N`), the HAProxy configuration is regenerated and rewritten to the VM disk, and then `docker kill -s HUP <container>` sends `SIGHUP` to the HAProxy master process. The master spawns new worker threads using the updated configuration while existing threads continue to drain active connections, achieving zero-downtime reconfiguration.

Schema self-healing. The `read_host_port` column required to store HAProxy’s read frontend port was added in a later development phase. Rather than requiring a manual database migration, `api/main.py` executes `ALTER TABLE db_instances ADD COLUMN read_host_port INTEGER` inside a `try/except` block on every startup, silently catching the error when the column already exists.

9.5 Concurrency Model

The primary performance challenge in our system is that querying the Docker daemon of a remote VM requires a full SSH handshake followed by a socket-level Docker API call, taking 0.5–1.5 seconds per VM. Querying N VMs sequentially would impose $O(N)$ latency on every list or search request.

We address this with Python’s `ThreadPoolExecutor`. Because SSH socket I/O releases the Global Interpreter Lock (GIL), multiple threads can perform network I/O truly concurrently. Three critical code paths use this approach:

- `get_all_clients` in `api/containers/docker_client.py`: opens Docker connections to all the user’s active VMs in parallel using `executor.map(connect_vm, instances)`.
- `list_containers` and `get_container`: once connections are open, container listing and search are also dispatched in parallel.
- `get_db_container_and_client` in `api/database_service/db_manager.py`: searches for a specific container ID across all VMs. To prevent multiple threads from both claiming the same container and leaking open sockets, a `threading.Lock()` guards a shared `found` dictionary. The first thread to locate the container acquires the lock, writes its `client` and `container` references, and releases the lock; subsequent threads detect that `found["container"]` is not `None` and immediately close their own client connection before returning.

Empirical measurements in a three-VM environment show that SSH connection setup reduces from 3.42 seconds sequentially to 1.15 seconds in parallel (approximately $3\times$ speedup), and container list queries reduce from 1.85 seconds to 0.65 seconds ($\approx 2.8\times$ speedup).

10 Web Dashboard

The web dashboard provides a browser-based interface for all platform capabilities. We built it as a React 18 [10] single-page application using the Vite build tool [11] and Tailwind CSS [12] for utility-first styling. The dashboard communicates exclusively with the FastAPI backend over REST, augmented by a WebSocket connection for the in-browser SSH terminal.

10.1 Application Structure

The entry point `dashboard/src/App.jsx` configures the React Router, wraps the router in a `QueryClientProvider` (TanStack React Query [13]), and declares six routes:

Route	Component
<code>/login</code>	<code>Login.jsx</code>
<code>/register</code>	<code>Register.jsx</code>
<code>/</code>	<code>Overview.jsx</code> (protected)
<code>/vms</code>	<code>VMs.jsx</code> (protected)
<code>/containers</code>	<code>Containers.jsx</code> (protected)
<code>/storage</code>	<code>Storage.jsx</code> (protected)

Protected routes are wrapped in a `ProtectedRoute` component that reads the JWT from `localStorage["cloud_token"]`. If no token is present, the user is immediately redirected to `/login`. This check is synchronous and client-side: the server enforces the same token on every API call, so any forged or expired token stored locally will result in a 401 response and a redirect to `/login` triggered by the Axios interceptor.

10.2 HTTP Client and JWT Interceptor

`dashboard/src/api/client.js` creates an Axios instance with `baseUrl` set to `http://localhost:8000`, the FastAPI development server address. A request interceptor is registered that reads `localStorage.getItem("cloud_token")` before every request and, if a token is present, injects it as an `Authorization: Bearer <token>` header:

```

1  apiClient.interceptors.request.use((config) => {
2      const token = localStorage.getItem("cloud_token");
3      if (token) {
4          config.headers.Authorization = `Bearer ${token}`;
5      }
6      return config;
7  });

```

Listing 18: Axios JWT request interceptor.

A response interceptor handles 401 Unauthorized responses by clearing the stored token `localStorage.removeItem("cloud_token")` and redirecting the browser to `/login`. This means that a user whose token expires mid-session is automatically signed out on their next API call, rather than receiving confusing permission-denied errors.

10.3 Authentication State Management

Token state is managed by a Zustand [14] store rather than React context, because Zustand’s store is a plain JavaScript singleton that does not require a Provider wrapper and does not trigger re-renders in components that do not subscribe to the changed slice. The auth store exposes `token`, `setToken(t)`, and `clearToken()`.

Authentication operations are defined in `dashboard/src/api/auth.js`:

- `login(username, password)`: POSTs to `/auth/login`, stores the returned `access_token` in both the Zustand store and `localStorage`, and navigates to `/`.
- `register(username, email, password)`: POSTs to `/auth/register`.
- `getMe()`: GETs `/auth/me`, returning the `UserResponse` including the SSH key.
- `updateMe(data)`: PUTs `/auth/me` with the `UserUpdate` payload.
- `deleteMe()`: DELETEs `/auth/me`, clears the token, and redirects to `/register`.

10.4 Real-Time Metrics with React Query

All server state (VMs, containers, cluster status, energy stats) is managed by TanStack React Query. Each data-fetching operation is defined as a `useQuery` call with a stable cache key and a `refetchInterval` of 10 seconds:

```

1  const { data: vms } = useQuery({
2      queryKey: ["vms"],
3      queryFn: () => apiClient.get("/compute/vms").then(r => r.data),
4      refetchInterval: 10000
5  });

```

Listing 19: Auto-refreshing VM list query.

This pattern means the dashboard stays up-to-date with the cluster state without requiring the user to manually reload pages. When the autoscaler scales VMs up or down in the background, the dashboard reflects the change within 10 seconds. React Query also handles loading and error states declaratively, so each component can conditionally render a spinner or an error message without imperative `useEffect` state management.

10.5 Overview Dashboard

`dashboard/src/pages/Overview.jsx` is the landing page after login. It renders:

- **StatCard components:** Four summary cards showing total VMs, running VMs, active containers, and total storage files. Each fetches its data via a dedicated `useQuery` hook.
- **CPU chart:** A `Recharts` [15] `AreaChart` plotting cluster average CPU percentage over time. Historical data points are fetched from `GET /compute/vms/{id}/metrics` for all active VMs; the 50 most recent readings per VM are averaged and rendered as a shaded area chart. The time axis displays the last 15 minutes of samples.
- **Autoscaler panel:** Displays the current autoscaler status, the `SCALE_UP_CPU_PCT` and `SCALE_DOWN_CPU_PCT` thresholds, and the `MIN_VMS/MAX_VMS` bounds from the `GET /compute/status` response.
- **Energy savings panel:** Fetches `GET /compute/energy-stats` and displays the total VM-hours saved, kilowatt-hours conserved, and estimated CO₂ avoided in a highlighted card with green colouring.
- **Quick actions:** Buttons for “Provision VM”, “Launch Container”, and “Upload File” that navigate to the corresponding subsystem page or open an inline modal.

10.6 VM Management Page

`dashboard/src/pages/VMs.jsx` lists the user’s VMs in a table with columns for name, state, CPU%, memory, and IP address. Clicking a VM row expands an inline `VMMetricsGraph` component that renders a `Recharts LineChart` of the VM’s historical CPU readings (the `vm_metrics` array from `GET /compute/vms/{id}/metrics`). Buttons in each row trigger `POST /compute/vms/{id}/stop` or `POST /compute/vms/{id}/start`, with `React Query`’s `useMutation` invalidating the `["vms"]` query cache on success to trigger an immediate refresh.

A “New VM” modal allows the user to select a template from the `GET /compute/templates` endpoint and submit a `POST /compute/vms` request. If a pre-warmed VM is available, the dashboard displays the provisioning as near-instant (< 1 second); otherwise, it shows a spinner with an estimated 90-second wait.

An embedded `WebSocket` terminal is accessible from each VM row. When the terminal icon is clicked, the component opens a native browser `WebSocket` connection to `ws://localhost:8000/terminal/{vm_ip}`. Keystrokes are sent as UTF-8 strings via `ws.send()`, and incoming data is appended to a scrollable `<pre>` element using a `useRef` to maintain scroll position. The simple `xterm.js`-style textarea allows users to run shell commands without leaving the browser.

10.7 Containers and Storage Pages

`Containers.jsx` provides a card grid of the user’s containers, each showing the image name, status, exposed ports, and host VM. A “Launch” form accepts an image name, optional environment key-value pairs, and a container port to expose. All container lifecycle operations (start, stop, remove, view logs) are implemented as `useMutation` hooks wrapping the corresponding `/containers` endpoints.

`Storage.jsx` renders two tabs: one for object files (with a drag-and-drop upload zone backed by the `POST /storage/files` endpoint and a sortable table of existing files with download and delete actions) and one for block devices (with a form to allocate a new `DATABLOCK` image, a list of existing disks with their state and size, and attach/detach actions that accept a VM ID as input).

11 Tests & Evaluation Metrics

All test outputs below are real, captured from live runs against the running cloud infrastructure.

Environment: OpenNebula 7.2.0 · FastAPI backend · Alpine Linux VMs · KVM/QEMU hypervisor

11.1 Quick Summary

#	Test	Script	Result
1	OpenNebula Connection Integrity	scripts/test_connection.py	PASS
2	Block Storage Lifecycle (Provision → Ready → Delete)	scripts/test_disks.py	PASS
3	Scale-to-Zero & Wake-on-API	scripts/test_scale_to_zero.py	PASS
4	HTTP Load Balancer CPU Stress Test	scripts/test_http_stress.py	PASS
5a	DB Cluster — Write Isolation (HAProxy write port)	scripts/test_db_cluster.py	PASS
5b	DB Cluster — Replicas Reject Writes (read-only)	scripts/test_db_cluster.py	PASS
5c	DB Cluster — Live WAL Replication	scripts/test_db_cluster.py	PASS*
5d	DB Cluster — Round-Robin Read Load Balancing	scripts/test_db_cluster.py	PASS

Table 5: Database Cluster and API Infrastructure Verification Summary

* WAL replication confirmed live via `pg_stat_replication` (2 replicas streaming async).

The 5-second automated window was too short after a VM reboot; direct SQL verification confirmed replication was active.

11.2 Test 1 — OpenNebula Connection Integrity

Script: `scripts/test_connection.py`

Purpose: Verifies connection and authentication with the OpenNebula XML-RPC endpoint (`oned`), queries active hosts, datastores, VMs, and templates.

How to run:

```
1 ssh -L 2633:localhost:2633 -L 9002:localhost:9002 ubuntu@[gateway-ip]
2 source .venv/bin/activate
3 python scripts/test_connection.py
```

Live Output:

```
1 =====
2 Testing OpenNebula connection...
3 =====
4
5 [OK] Connected! OpenNebula version: 7.2.0
6
7 [OK] Users in OpenNebula:
8   - [2] angie
9   - [1] serveradmin
10  - [0] oneadmin
11
12 [OK] Virtual Machines (3 found):
13   - [83] prewarmed-vm-1780509189    state=3
14   - [75] ola                        state=8
15   - [66] auto-vm-angie-1780325875  state=8
16
17 [OK] VM Templates (2 found):
18   - [2] UBUNTU 24.04
19   - [0] Alpine Linux 3.20
20
21 [OK] Datastores (3 found):
```

```

22 - [2] files      type=2
23 - [1] default   type=0
24 - [0] system    type=1
25
26 [OK] Hosts / Physical machines (1 found):
27 - [0] localhost state=2
28
29 =====
30 Phase 1 connection test complete.
31 =====

```

Result: **PASS**

Check	Outcome
XML-RPC connection to OpenNebula 7.2.0	PASS
User pool enumeration (3 users)	PASS
VM pool listing (3 VMs found)	PASS
Template pool listing (2 templates)	PASS
Datastore listing (3 datastores)	PASS
Physical host listing (1 KVM host)	PASS

Table 6: OpenNebula Connection Integrity Check List

11.3 Test 2 — Block Storage Lifecycle

Script: scripts/test_disks.py

Purpose: Validates the full lifecycle of a block storage datablock — provisioning a 1 GB image in OpenNebula, waiting for READY state, then deleting it cleanly via the REST API.

How to run:

```

1 python -m uvicorn api.main:app --host 0.0.0.0 --port 8000
2 source .venv/bin/activate
3 python scripts/test_disks.py

```

Live Output:

```

1 =====
2 Testing Disk Management (Block Storage) API...
3 =====
4
5 [1/4] Provisioning a custom 1GB disk: test-disk-1780916759...
6 [OK] Disk provisioned! local_id: 3, one_image_id: 7, status: LOCKED
7
8 [2/4] Listing user disks...
9 - Disk [3] test-disk-1780916759 (1 GB) - Status: LOCKED
10 [OK] Disk found in user disk list.
11
12 [3/4] Waiting for disk to become READY in OpenNebula...
13 Check 1: status is 'READY'
14 [OK] Disk is READY!
15
16 [4/4] Deleting disk 3...

```

```

17 [OK] Disk successfully deleted!
18
19 =====
20 Disk Management API test complete.
21 =====

```

Result: **PASS**

Step	Endpoint	Outcome	Timing
Provision 1 GB disk	POST /storage/disks	PASS	< 1s
Appears in listing	GET /storage/disks	PASS	immediate
OpenNebula image READY	internal poll	PASS	< 1.5s
Clean deletion	DELETE /storage/disks/{id}	PASS	< 1s

Table 7: Block Storage Lifecycle Operation Metrics

11.4 Test 3 — Scale-to-Zero & Wake-on-API

Script: scripts/test_scale_to_zero.py

Purpose: Validates the full energy-saving lifecycle — VM auto-provisioning on demand, automatic suspension after user inactivity, and transparent VM resumption on user activity.

How to run:

```

1 source .venv/bin/activate
2 python scripts/test_scale_to_zero.py

```

Live Output:

```

1 =====
2 STEP 1: Verify on-demand VM booting / creation
3 =====
4 Ensuring user 'angie' has a running VM...
5 DEBUG: VM 'auto-vm-angie-1780325875' is POWEROFF. Resuming...
6 DEBUG: Waiting for VM to boot... State=ACTIVE, LCM=20 (attempt 1/45)
7 DEBUG: Waiting for VM to boot... State=ACTIVE, LCM=20 (attempt 2/45)
8 DEBUG: VM is booted but Docker/SSH not ready yet: BrokenPipeError
9 DEBUG: Triggering self-healing for VM at IP 172.16.100.2...
10 DEBUG: Waiting for VM to boot... State=ACTIVE, LCM=3 (attempt 3/45)
11 DEBUG: VM 'auto-vm-angie-1780325875' is fully RUNNING. Docker reachable.
12 [OK] ensure_user_has_running_vm returned VM ID: 38
13 VM state: Name=auto-vm-angie-1780325875, State=ACTIVE, LCM=3, IP=172.16.100.2
14
15 =====
16 STEP 2: Verify user inactivity VM suspension
17 =====
18 Overriding inactivity timeout to 5 seconds...
19 Simulating user inactivity. last_active_at set to 10s ago.
20 Running autoscaler loop to detect inactivity...
21 Waiting for VM to transition to SUSPENDED or POWEROFF (polling up to 60s)...
22 Poll 1: VM State=POWEROFF
23 [OK] VM 'auto-vm-angie-1780325875' was automatically suspended due to inactivity.

```

```

24
25 =====
26 STEP 3: Verify user activity VM resumption
27 =====
28 Simulating user activity (updating last_active_at to now)...
29 Running autoscaler loop to detect activity...
30 Waiting for VM to transition back to ACTIVE (polling up to 60s)...
31 Poll 1: VM State=ACTIVE, LCM=20
32 [OK] VM 'auto-vm-angie-1780325875' was automatically resumed.
33 Waiting for VM to be fully RUNNING again...
34 [OK] VM is fully booted and ready.
35
36 =====
37 TEST SUMMARY: ALL PASSED!
38 =====
39 1. [PASS] On-demand VM provisioning/booting works.
40 2. [PASS] Inactive user VMs automatically suspend to save energy.
41 3. [PASS] Active user VMs automatically resume.
42
43 SLA thresholds successfully restored.

```

Result: **PASS** — All 3 steps passed

Step	Action	Outcome
Step 1	VM auto-provisioned; Docker verified reachable	PASS
Step 2	VM auto-suspended after 5s simulated inactivity	PASS
Step 3	VM auto-resumed after simulated user activity	PASS

Table 8: Energy-saving Scale-to-Zero and Wake-on-API Milestones

Key autoscaler log entries captured during test:

```

1 INFO:autoscaler: AutoScaler check -- active=1 total=3 avg_cpu=75.9%
2 INFO:autoscaler: Suspending VM 'auto-vm-angie-1780325875' (one_vm_id=66)
3 due to user 'angie' inactivity (idle for 12.2s)
4 INFO:autoscaler: Successfully claimed pre-warmed VM 'autoscale-vm-1780916784'
5 (one_vm_id=83) for autoscale (user_id=2)
6 INFO:autoscaler: Replenishing Pre-Warming Pool -- created standby VM
7 'prewarmed-vm-1780916794' (one_vm_id=84)
8 INFO:autoscaler: Resuming VM 'auto-vm-angie-1780325875' (one_vm_id=66)
9 because user 'angie' is active

```

11.5 Test 4 — HTTP Load Balancer CPU Stress Test

Script: scripts/test_http_stress.py

Purpose: Proves the Nginx load balancer distributes CPU load evenly across worker containers by deploying 2 Nginx workers with gzip compression level 9 + a 4 MB compressible file, then hammering the LB with 30 concurrent curl loops for 20 seconds.

How to run:

```

1 source .venv/bin/activate
2 python scripts/test_http_stress.py

```

Live Output:

```

1 =====
2 LB CPU Stress Test -- concurrent requests through LB
3 =====
4
5 [PRE-TEST] Cleanup (scale to 0)...
6 * Cleaned up.
7
8 [STEP 1] Provisioning 2 workers + LB (no_autoscale=True)...
9 * Done in 214.7s -- LB: http://172.16.100.3:32770
10 Worker 0: 50eac7bab4c3 on VM 172.16.100.3
11 Worker 1: 573f48997970 on VM 172.16.100.3
12 LB URL (from VM 172.16.100.3): http://127.0.0.1:32770/large.txt
13
14 [STEP 2] Waiting for containers to be running...
15 * Worker 0: 'angie-test-web-lb-worker-...-0' running.
16 * Worker 1: 'angie-test-web-lb-worker-...-1' running.
17
18 [STEP 3] Deploying gzip config + 4 MB test file on each worker...
19 * Worker 0 configured.
20 * Worker 1 configured.
21
22 [STEP 4] Connectivity check...
23 * LB -> workers responded HTTP 200
24
25 [STEP 5] Baseline CPU (before flood)...
26 Worker 0 (172.16.100.3): 0.0%
27 Worker 1 (172.16.100.3): 0.0%
28
29 [STEP 6] Flooding LB with 30 concurrent curl loops for 20s...
30 * Waiting 5s before CPU sample 1/3...
31 Worker 0 (172.16.100.3): 36.2% CPU
32 Worker 1 (172.16.100.3): 34.9% CPU
33 * Waiting 5s before CPU sample 2/3...
34 Worker 0 (172.16.100.3): 0.0% CPU (flood ended)
35 Worker 1 (172.16.100.3): 0.0% CPU
36
37 [STEP 7] Post-flood CPU...
38 Worker 0: 0.0%
39 Worker 1: 0.0%
40
41 [STEP 8] Request distribution (nginx docker logs)...
42 Worker 0 (172.16.100.3): 239 requests
43 Worker 1 (172.16.100.3): 238 requests
44
45 =====
46 RESULTS SUMMARY
47 =====

```

```

48
49 Request Distribution:
50 Worker 0      239 requests    50.1% [OK]
51 Worker 1      238 requests    49.9% [OK]
52 Total: 477 requests handled
53 [OK] LB distributed traffic to ALL workers.
54
55 CPU under flood:
56 Worker 0      baseline 0.0%   peak 36.2%   delta +36.2% [OK]
57 Worker 1      baseline 0.0%   peak 34.9%   delta +34.9% [OK]
58 [OK] PASS -- CPU spiked on ALL workers via LB requests.
59
60 [CLEANUP] Scaling to 0...
61 [OK] Done.
62 =====

```

Result: **PASS**

Metric	Value
Total requests handled	477
Worker 0 share	239 req (50.1%)
Worker 1 share	238 req (49.9%)
Worker 0 CPU peak under load	36.2% (baseline 0.0%)
Worker 1 CPU peak under load	34.9% (baseline 0.0%)
Post-flood CPU	0.0% (both)
LB distribution	Near-perfect 50/50 round-robin

Table 9: Web Layer Load Balancer Stress Metrics

11.6 Test 5 — Clustered Database & HAProxy L4 Load Balancing

Script: scripts/test_db_cluster.py

Purpose: Validates three core guarantees of the PostgreSQL DBaaS cluster — write isolation via HAProxy, live WAL streaming replication, and round-robin read distribution.

Cluster at test time:

```

1 Primary:      172.16.100.2   container=db-angie-test-db-lb-primary
2 HAProxy:      172.16.100.2   write port=32772   read port=32771
3 Replica #1: 172.16.100.5   container=db-angie-test-db-lb-replica-1
4 Replica #2: 172.16.100.5   container=db-angie-test-db-lb-replica-2

```

Test 5a & 5b — Write Isolation

```

1 1a. INSERT via HAProxy write port (should reach primary only)...
2 [OK] INSERT via write port succeeded (primary accepted the write).
3 Output:
4 CREATE TABLE
5 TRUNCATE TABLE
6 INSERT 0 1
7 Queried row message from primary: 'initial-write'
8
9 1b. INSERT directly on each replica (should be rejected as read-only)...

```

```

10 [OK] Replica #1 (172.16.100.5) correctly rejected the write.
11     PostgreSQL error: ERROR: cannot execute INSERT in a read-only transaction
12 [OK] Replica #2 (172.16.100.5) correctly rejected the write.
13     PostgreSQL error: ERROR: cannot execute INSERT in a read-only transaction

```

Sub-test	Outcome
INSERT via HAProxy write port → primary	Accepted
INSERT directly to Replica #1	Rejected (read-only)
INSERT directly to Replica #2	Rejected (read-only)

Table 10: Dual-port Cluster Write Path Separation Verification

Test 5c — Live WAL Replication

WAL streaming confirmed via direct SQL after updating `primary_conninfo` on replicas (port changed after VM reboot):

```

1 -- SELECT client_addr, state, sync_state FROM pg_stat_replication;
2 client_addr | state | sync_state
3 -----+-----+-----
4 172.16.100.5 | streaming | async
5 172.16.100.5 | streaming | async
6 (2 rows)

1 -- Replica-1 container log after reconnect:
2 LOG: parameter "primary_conninfo" changed to "host=172.16.100.2 port=32768"
3 LOG: started streaming WAL from primary at 0/5000000 on timeline 1
4 LOG: restartpoint complete: wrote 14 buffers (0.1%); distance=25 kB

```

Both replicas confirmed in **async streaming** state, continuously receiving WAL changes.

Test 5d — Round-Robin Read Load Balancing

```

1 Sending 8 SELECT queries to the read port (32771)...
2 Each query reports which backend PostgreSQL container answered it.
3
4 Query # 1 -> backend: 65d3f4c3a82c (Replica #1)
5 Query # 2 -> backend: 6ecc7d07be90 (Replica #2)
6 Query # 3 -> backend: aee598c76def (Primary)
7 Query # 4 -> backend: 65d3f4c3a82c (Replica #1)
8 Query # 5 -> backend: 6ecc7d07be90 (Replica #2)
9 Query # 6 -> backend: aee598c76def (Primary)
10 Query # 7 -> backend: 65d3f4c3a82c (Replica #1)
11 Query # 8 -> backend: 6ecc7d07be90 (Replica #2)
12
13 Unique backends that responded: ['65d3f4c3a82c', '6ecc7d07be90',
14     'aee598c76def']
15 [OK] 3 different backends responded -> round-robin is WORKING.

```

11.7 Evaluation Metrics

11.7.1 Metric A — VM Provisioning Latency: Cold Boot vs. Pre-Warmed Standby Pool

Improvement factor: $\approx 122x$ faster provisioning

Queries	Container ID	Node
1, 4, 7	65d3f4c3a82c	Replica #1
2, 5, 8	6ecc7d07be90	Replica #2
3, 6	aee598c76def	Primary

Table 11: Round-robin Read Distribution via HAProxy Port 5433

Provisioning Method	Average Latency	Mechanism
Cold Boot (from scratch)	≈ 97.5 s	Disk clone + kernel boot + context init
Pre-Warmed Standby Pool	< 1 s (0.8 s)	Owner transfer (chown) + rename only

Table 12: VM Provision Latencies: Cold vs Standby

11.7.2 Metric B — API Monitoring Latency: Sequential vs. Parallel Thread Pool

Monitoring Strategy	Latency (3 VMs)	Implementation
Sequential	≈ 3.42 s	for vm in active_vms: probe(vm)
Parallel (ThreadPoolExecutor)	≈ 1.15 s	ThreadPoolExecutor.map(probe, active_vms)

Table 13: Monitoring Threading Strategy Query Latency

Improvement: $\approx 3x$ speedup (66% latency reduction)

11.7.3 Metric C — Energy & Carbon Savings via Scale-to-Zero

$$\begin{aligned} \text{Hours Saved} &= (N_{\max} \times T_{\text{elapsed}}) - \sum (\text{Active VM runtimes}) \\ \text{Energy Saved [kWh]} &= \frac{\text{Hours Saved} \times P_{\text{avg}}}{1000} \\ \text{CO}_2 \text{ Saved [kg]} &= \text{Energy Saved} \times EF \end{aligned}$$

Where:

- $N_{\max} = 5$ VMs (maximum pool capacity)
- $P_{\text{avg}} = 50$ W (estimated host power per active VM)
- $EF = 0.4$ kg/kWh (EU average grid carbon emission factor)

Concrete 24-hour example (observed student workload):

Parameter	Value
Elapsed period	24 hours
Active user sessions	2 users
Total VM active runtime	≈ 6 hours
Baseline (always-on, 5 VMs)	120 VM·hours
Hours saved	$120 - 6 = 114$ VM·hours
Energy saved	$(114 \times 50) / 1000 = 5.7$ kWh
CO ₂ saved	$5.7 \times 0.4 = 2.28$ kg CO₂

Table 14: Estimated Energy/CO2 Savings over 24-Hour Period

VM Name	IP Address	Role
auto-vm-angie-1780325875	172.16.100.2	Tenant VM / Primary DB host
prewarmed-vm-1780509189	standby pool	Pre-warmed standby (unclaimed)
autoscale-vm-1780916784	172.16.100.3	Autoscale worker (LB stress test)
DB replica VM	172.16.100.5	Hosts Replica #1 and Replica #2

Table 15: VM Network Node Infrastructure Map

11.8 Infrastructure State at Test Time

11.8.1 VM Network Layout

11.8.2 OpenNebula Cluster Specs

- **Version:** 7.2.0
- **Templates:** Alpine Linux 3.20 (primary base), UBUNTU 24.04
- **Datastores:** system (type=1), default (type=0), files (type=2)
- **Hosts:** 1 KVM hypervisor (localhost, state=MONITORED)

12 Conclusion

In this project we designed and implemented a full-stack, multi-tenant cloud management platform that spans from the hypervisor layer up to a browser-based dashboard. The system encompasses eight distinct service subsystems , virtual machine provisioning, block storage, object storage, container management, Database-as-a-Service, load balancing, autoscaling, and authentication , all integrated through a single FastAPI REST API and a React single-page application.

12.1 Engineering Achievements

Several of the engineering challenges we encountered required solutions that go beyond standard tutorial patterns.

Sub-second VM provisioning via pre-warming. The cold-boot cycle for an Alpine Linux VM on OpenNebula takes 85–110 seconds, which is unacceptably slow for interactive provisioning. We solved this by maintaining a pool of pre-warmed standby VMs: the autoscaler allocates a VM under system ownership and names it with a `prewarmed-vm-` prefix. When a user requests a new VM, the API checks for an available standby first. If found, a rename (`client.vm.rename`) and ownership transfer (`client.vm.chown`) complete in under one second because no hypervisor operation is required , the VM is already running. This reduces perceived provisioning latency by two orders of magnitude.

Accurate Docker CPU measurement. Docker’s single-snapshot `stats(stream=False)` interface returns a stale `precpu_stats` baseline and therefore always yields 0% CPU on the first call. We identified this behaviour during testing and implemented a two-snapshot approach: the autoscaler takes a first snapshot, sleeps one second, takes a second snapshot, and computes the delta. This produces accurate CPU measurements even at very low load levels and is essential for the container autoscaler’s scale-down decisions.

Concurrent VM querying with first-win locking. Searching for a container across multiple VMs requires opening an SSH connection and issuing a Docker API call to each VM. Done sequentially, this is $O(N)$ in latency. We decomposed the problem into parallel sub-tasks using `ThreadPoolExecutor` and used a `threading.Lock` to implement a first-win pattern: the thread that locates the container first stores the result under the lock,

and all other threads detect the result and immediately release their SSH connections. This approach delivers approximately $3 \times$ speedup compared to sequential execution while avoiding connection leaks.

HAProxy dynamic reconfiguration. Changing an HAProxy configuration without dropping active connections requires sending `SIGHUP` to the master process after updating the configuration file on disk. Our implementation writes the new `haproxy.cfg` to the container’s mounted directory via SFTP (`write_ssh_file` in `api/loadbalancer/ssh_utils.py`) and then issues `docker kill -s HUP <container>` through an SSH channel. The HAProxy master spawns new worker threads with the updated configuration while existing workers finish draining their active connections. This hot-reload mechanism allows us to add or remove PostgreSQL replicas without any downtime to the database cluster.

Automated service auto-recovery. In our elastic compute model, suspending or rebooting a tenant VM causes Docker to terminate running containers and reassign dynamic host ports upon restart. This breaks both outer HAProxy load balancer configurations and replica database connection targets. We resolved this by implementing an auto-recovery subsystem within the autoscaler loop. Resumed VMs are tracked in a thread-safe queue. Once online, the system automatically starts exited containers, queries the new port bindings, synchronizes SQLite metadata, regenerates and hot-reloads the HAProxy configuration, and re-binds replica databases to the new primary port via `ALTER SYSTEM` queries. This guarantees self-healing and zero manual intervention during scale-to-zero and wake-on-API cycles.

Schema self-healing migrations. Because the project was developed iteratively, new database columns were occasionally required in later phases. Rather than maintaining a formal migration framework (such as Alembic), we used a self-healing pattern in `api/main.py`: each new column is added via an `ALTER TABLE ... ADD COLUMN` statement inside a `try/except OperationalError` block that silently ignores the error if the column already exists. This approach is trivially correct for append-only schema evolution and requires no migration state management.

Multi-layer isolation without a dedicated identity service. Our multi-tenancy model enforces isolation simultaneously at the JWT layer (token expiry and signature verification), the SQL layer (foreign-key user-id filters), the OpenNebula layer (chown at provisioning time), and the label/bucket namespacing layer (Docker labels and MinIO bucket names). This defence-in-depth approach means that a compromise of any single layer does not immediately expose another tenant’s resources.

12.2 Limitations and Future Work

Despite its breadth of functionality, the current implementation carries several limitations that would need to be addressed before production deployment.

SQLite as the metadata store. SQLite does not support concurrent writes from multiple processes, so the current system requires a single API server process. Scaling the FastAPI backend horizontally (e.g., with Gunicorn workers or a Kubernetes deployment) would require migrating the metadata store to a networked RDBMS such as PostgreSQL, possibly using Alembic for schema management.

Plaintext SSH credentials in environment variables. The gateway credentials (`GATEWAY_IP`, `GATEWAY_USER`, `GATEWAY_PASSWORD`) are loaded from a `.env` file. In a production system, these secrets should be managed by a secrets vault (HashiCorp Vault, AWS Secrets Manager, or Kubernetes Secrets) with short-lived leases and audit logs.

No persistent WebSocket terminal session. The WebSocket terminal connection is dropped on browser navigation or network interruption. Adding tmux or screen session persistence on the VM side (by attaching to a named session rather than spawning a new shell) would allow users to reconnect to running processes after disconnection.

Container autoscaler limited to uniform scaling policies. The current container autoscaler uses a flat per-group dictionary with system-wide `MAX_REPLICAS` and CPU threshold constants. If a user runs multiple independent scale groups, they share the same thresholds. Exposing per-group autoscaling policies (via a database-backed configuration model) would enable more flexible deployment scenarios.

Lack of a service discovery mesh. The current implementation relies on the central FastAPI orchestrator to dynamically query Docker ports and rewrite HAProxy/Nginx configurations over SFTP. In larger clusters, this central polling becomes a bottleneck. A valuable direction for future work would be replacing this centralized configuration management with a distributed service discovery registry (such as Consul, etcd, or ZooKeeper) or a lightweight service mesh. Containers would self-register their dynamic ports upon boot, and the load balancers would watch the registry for real-time upstream target updates, removing SSH/SFTP write dependencies.

Primary database scaling I/O impact. Our Database-as-a-Service model uses physical replication initialized via `pg_basebackup`. Because this copies the entire data directory from the primary over the network, scaling up read-replicas causes significant read I/O load on the primary node. For databases larger than a few gigabytes, this would degrade live transaction performance. Future work should implement continuous WAL archiving to our MinIO object storage. New database replicas could then be provisioned by restoring a backup directly from MinIO and replaying only the remaining WAL stream from the primary, enabling safe database autoscaling with zero impact on primary node performance.

Energy accounting is approximate. The 50 W/VM draw and 0.4 kg CO₂/kWh factors are illustrative constants derived from typical server hardware and European grid emission averages. A production implementation would use real-time power telemetry from the hypervisor host's BMC (Baseboard Management Controller) and live grid intensity data from an electricity map API.

12.3 Closing Remarks

This project demonstrates that a feature-competitive cloud management platform, spanning IaaS, CaaS (Container-as-a-Service), DBaaS, object storage, and PaaS-style autoscaling, can be implemented as a relatively compact Python codebase (approximately 3000 lines of backend logic) when built on top of well-designed open-source primitives: OpenNebula for hypervisor orchestration, the Docker SDK for container management, MinIO for object storage, HAProxy for load balancing, and FastAPI for API design. The engineering effort was concentrated not in inventing new infrastructure but in correctly composing existing tools, handling their failure modes, and enforcing consistent multi-tenant isolation across all layers.

References

- [1] OpenNebula Systems. OpenNebula cloud management platform. <https://opennebula.io>, 2024. Version 7.2.0.
- [2] Sebastián Ramírez Tiangolo. FastAPI: Modern, fast (high-performance) web framework for building apis with python. <https://fastapi.tiangolo.com>, 2024. Version 0.111.0.
- [3] MinIO Inc. MinIO: High performance object storage. <https://min.io>, 2024. S3-compatible object storage server.

- [4] HAProxy Technologies. HAProxy: The reliable, high performance tcp/http load balancer. <https://www.haproxy.org>, 2024. Version 2.8.
- [5] OpenNebula Systems. pyone: Python bindings for the OpenNebula XML-RPC api. <https://github.com/OpenNebula/pyone>, 2024.
- [6] Michael Bayer. SQLAlchemy: The database toolkit for python. <https://www.sqlalchemy.org>, 2024. Version 2.0.
- [7] Michael Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). RFC 7519, Internet Engineering Task Force, 2015. <https://datatracker.ietf.org/doc/html/rfc7519>.
- [8] Jeff Stafford and Paramiko contributors. Paramiko: A python implementation of SSHv2. <https://www.paramiko.org>, 2024.
- [9] Docker Inc. Docker SDK for python. <https://docker-py.readthedocs.io>, 2024.
- [10] Meta Platforms, Inc. React: The library for web and native user interfaces. <https://react.dev>, 2024. Version 18.
- [11] Evan You and Vite contributors. Vite: Next generation frontend tooling. <https://vitejs.dev>, 2024. Version 5.
- [12] Adam Wathan and Tailwind Labs. Tailwind CSS: A utility-first CSS framework. <https://tailwindcss.com>, 2024. Version 3.
- [13] Tannerlinsley and TanStack contributors. TanStack query (formerly react query). <https://tanstack.com/query>, 2024.
- [14] Daishi Kato and Zustand contributors. Zustand: A small, fast, and scalable state management solution. <https://github.com/pmndrs/zustand>, 2024.
- [15] Recharts contributors. Recharts: Redefined chart library built with React and D3. <https://recharts.org>, 2024.